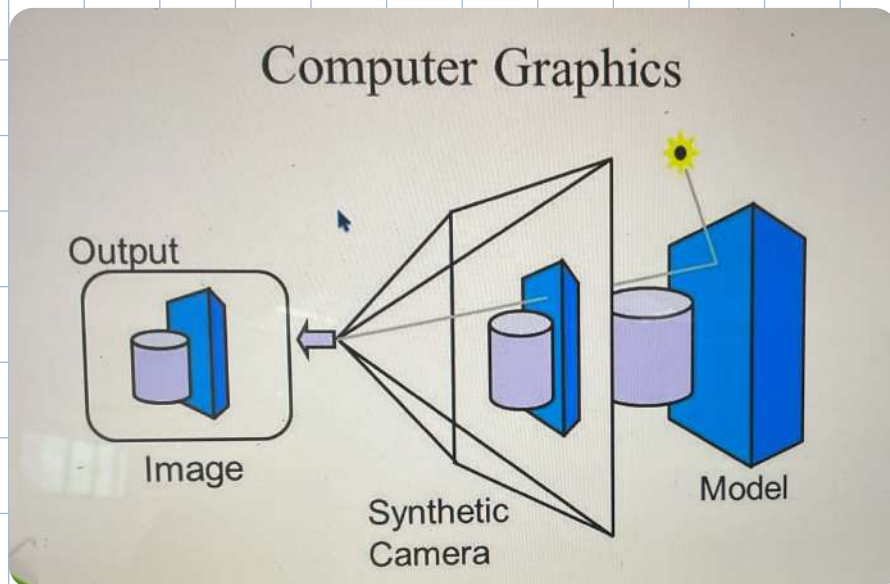


COMPUTER GRAPHICS:

→ deals with all aspects of creating images with a computer.

→ deals with:

- (i) Geometric modelling - creating mathematical models of 2D and 3D objects
- (ii) Rendering - producing images of the objects
- (iii) Animation - representing time dependent behaviour of objects



⇒ APPLICATIONS OF COMPUTER GRAPHICS:

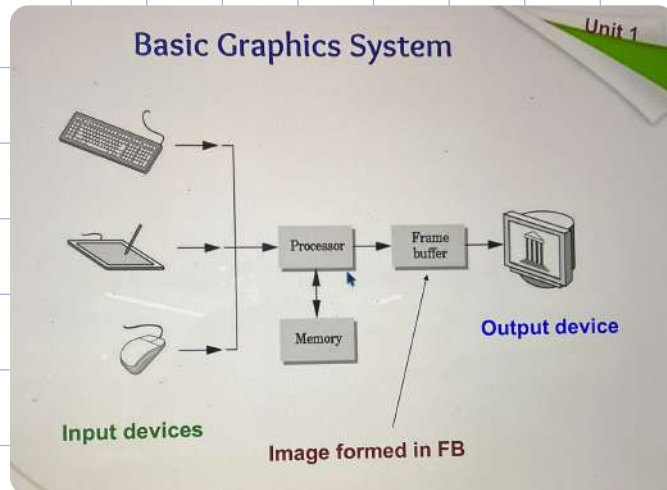
- (1) Display of info
- (2) Design
- (3) Simulation and animation
- (4) User Interfaces

A GRAPHICS SYSTEM:

- a graphics system is specialized for performing graphic based operations and image generations.

- 5 major elements:

- (1) input devices
- (2) processor
- (3) memory
- (4) Frame buffer
- (5) Output devices



★ All graphic systems have a specialized graphics processor. It is designed to carry out special graphics functions such as rasterization (Scan Conversion)

↳ process of converting primitives into pixel representation (2D Image)

- each point of this image contains information such as color and depth.

→ 2 types of memory:

- (1) General Memory - used to store non-graphics based data
- (2) Frame Buffer - used to store graphics data

⇒ Frame Buffer:

- Special type of memory present on graphics system.

- in CG, a picture or an image is produced as an array of pixels (Raster). These pixels are stored in the frame buffer.

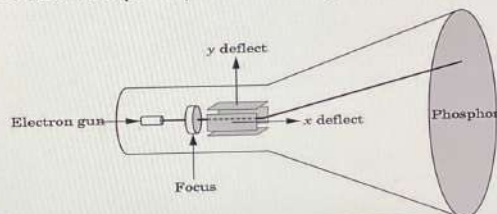
→ Resolution of frame buffer: No. of pixels in the frame buffer

→ Depth (precision of frame buffer): no. of bits of each pixel
↓
determines properties like how many colors can be represented
(2^n)

→ If depth = 24 bits / pixel or more → FULL COLOR (or TRUE COLOR (or) RGB COLOR system.

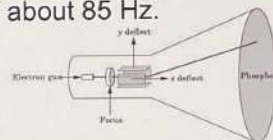
Working of Cathode Ray tube (CRT)

- The digital output produced by a processor is given to **digital-to-analog converter**.
- The analog voltages produced are applied across the x and y deflection plates.
- The direction of the beam can be controlled by these two deflection plates.
- When electrons strike the phosphor coating on the tube, light is emitted.



Working of Cathode Ray tube (CRT)

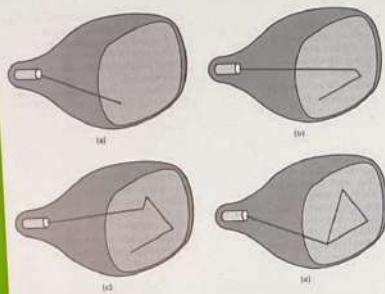
- Colored CRTs have 3 different colored phosphors (**RED, GREEN, BLUE**). They have 3 electron beams corresponding to 3 types of phosphors.
- A typical CRT will emit light only for a short duration when hit by an electron.
- This would produce flicker in the image.
- To produce a flicker-free image, the electron beam must be refreshed at a high rate called **refresh rate**.
- Modern displays have refresh rate of about 85 Hz.



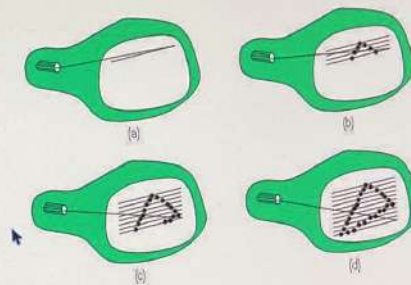
Random Scan CRT	Raster system CRT
Also called as calligraphic OR vector CRT.	Entire screen is a matrix of pixels.
Characters are made of sequence of strokes.	It takes pixels from frame buffer and displays them as points on the surface of the CRT. (Noninterlaced & interlaced displays)
A beam can be moved from any position to any other position.	Line cannot be drawn directly from one point to another.
Limited use.	Widely used.
Less efficient.	More efficient and can generate any image.

Types of CRTs

Random scan CRT



Raster system CRT

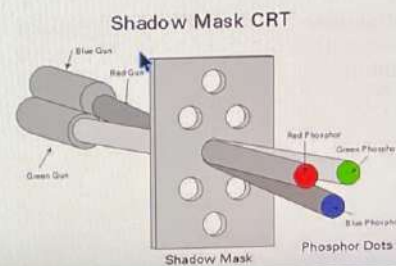
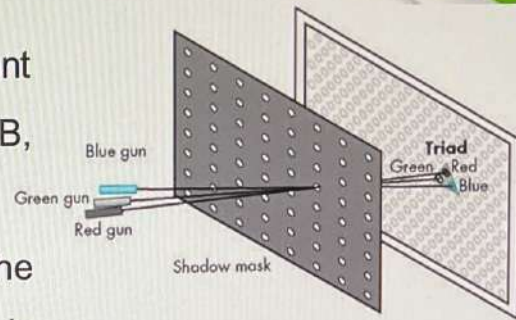


Shadow-mask CRT – colored CRT

The shadow mask is one of the technologies used to manufacture cathode ray tube (CRT) televisions and computer displays that produce color images.

Shadow-mask CRT – colored CRT

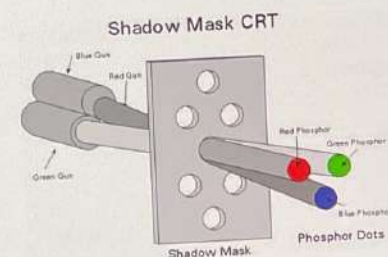
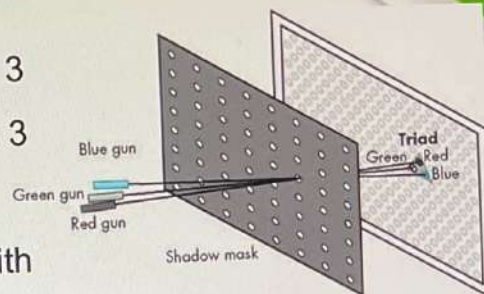
- Color CRTs have 3 different colored phosphors – RGB, arranged in small groups.
- A common style that arranges the phosphors in triangular groups is called as **Triads**, each triad consisting of 3 phosphors one for each primary.



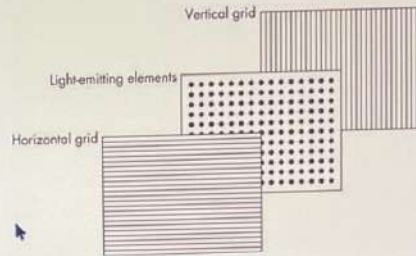
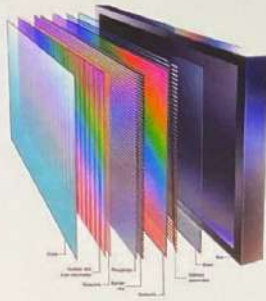
Shadow-mask CRT – colored CRT

Most of the color CRTs have 3 electron beams corresponding to 3 types of phosphors.

- Shadow mask, a metal screen with small holes (shadow mask), to ensure that an electron beam excites only phosphors of the proper color.



Generic flat-panel display

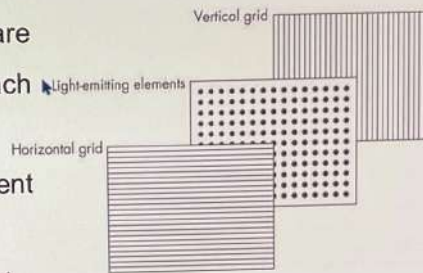


Popular displays

- LED
- LCD
- Plasma panels

Generic flat-panel display

- The two outside plates contain parallel grids of wires that are oriented perpendicular to each other.
- The middle panel is different for the 3 types of displays.
- LED, LCD and Plasma panel.



Generic flat-panel display

LED display

The middle panel contains Light emitting Diodes which are turned ON and OFF by the electrical signals sent to the grid.

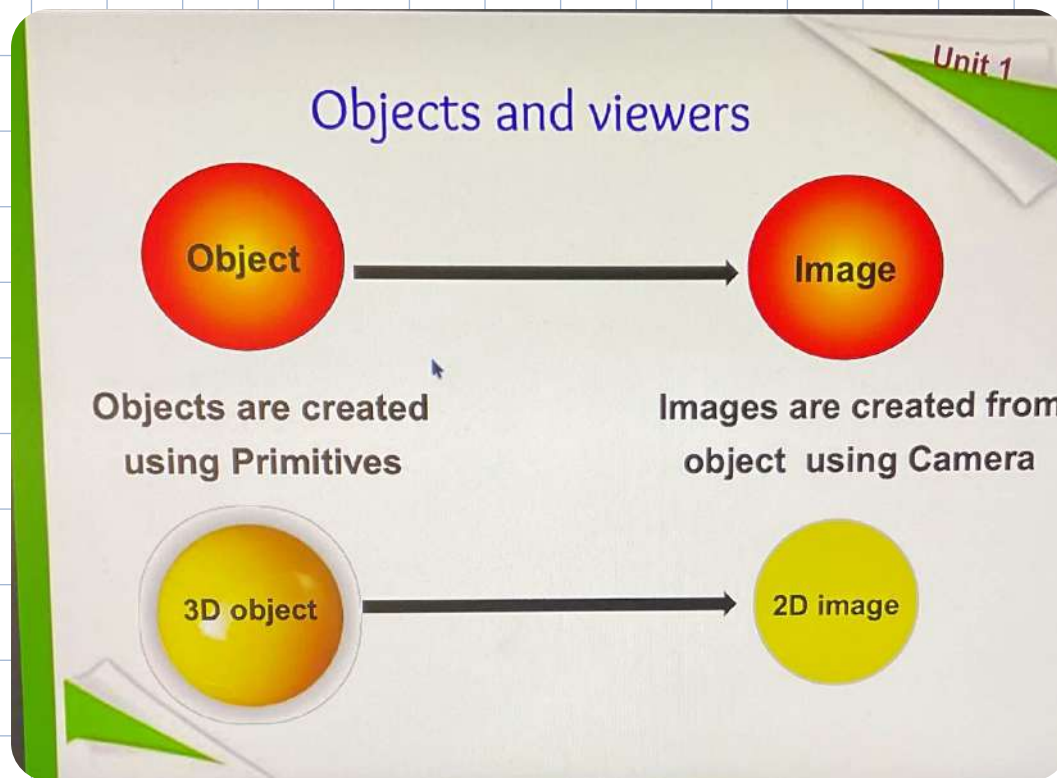
LCD display

Electrical field controls the polarization of liquid crystals in the middle panel.

Plasma

Uses voltage on the grids to energize gases embedded between the glass panels holding the grids. The energized gas becomes glowing plasma.

OBJECTS AND VIEWERS:



Basic entities that are part of any image formation process:

- (1) Objects
- (2) Viewers

→ Object:

- Physical structure of the image
- in graphics, we form objects by specifying the positions in space of various geometric primitives (lines, points, polygons)

A sample OpenGL code that specifies triangular polygon.

```
glBegin(GL_POLYGON);  
glVertex3f(0.0,0.0,0.0);  
glVertex3f(0.0,1.0,0.0);  
glVertex3f(1.0,0.0,0.0);  
glEnd();
```

→ Viewer:

- ↳ one who forms image of the object
- ↳ Obj's exist in real world and it is 3D. A viewer sees an object as image in 2D
- ↳ Viewer can be human, camera or digitizer

→ Image Formation:

It is the process by which the specification of the object is combined with the specification of the viewer to produce 2D image

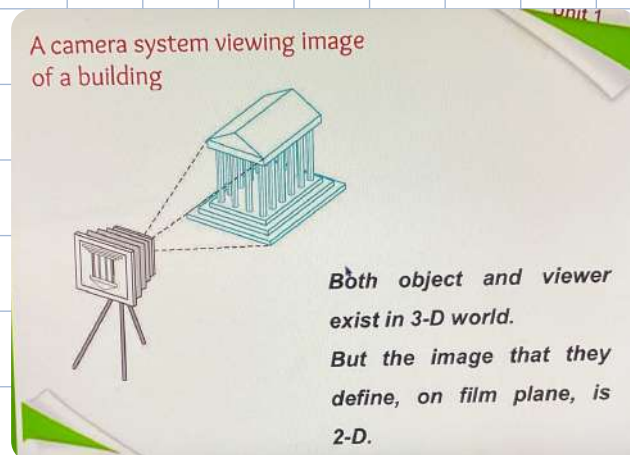
IMAGING SYSTEM:

- any obj is 3D.
- Image is always 2D
- so an obj from 3D is converted into image in 2D representation
- Elements of Image Formation:

(1) Object

(2) Camera (viewer)

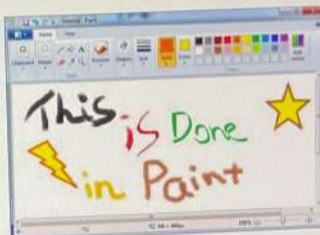
(3) Light source



THE PROGRAMMER'S INTERFACE (PI/API):

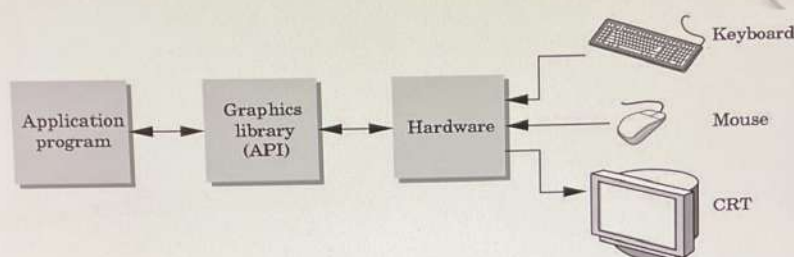
Interaction in paint program

- User develops images through interactions with the menus and icons using input devices.
- By clicking the menus or icons, the user guides the software and produces images without having to write programs.



- the interface b/w an application program and a graphics system can be specified through a set of functions that resides in a graphics library
- these specifications are called the application programmer's interface

The programmer's interface



- The application programmer writes an application program using graphics functions supported by the API.
- The output of the API is given to the software drivers which convert the data to a form understood by the particular hardware.

⇒ 3D APIs:

- Synthetic camera model is the basis for all popular APIs including OpenGL, Direct3D and Open Scene Graph.
- the model gives APIs to specify the following:
 - (1) object
 - (2) viewer
 - (3) light sources
 - (4) material properties

Objects in OpenGL

- Defined by **"set of vertices"**.
- Example : A rectangle can be created in OpenGL using 4 vertices.
- In graphics, we form objects by specifying the positions in space of various geometric primitives (lines, points, polygons).

Objects in OpenGL

OpenGL programs define primitives through **list of vertices**.
Ex., triangular polygon

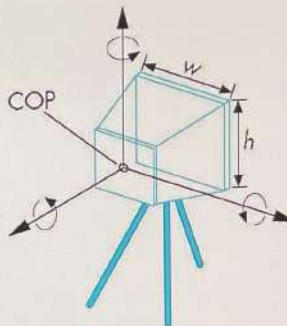
```
glBegin(GL_POLYGON);  
    glVertex3f(0.0, 0.0, 0.0); /* vertex A */  
    glVertex3f(0.0, 1.0, 0.0); /* vertex B */  
    glVertex3f(0.0, 0.0, 1.0); /* vertex C */  
glEnd();
```



Viewer or camera

A viewer can be specified in a variety of ways. The 4 types of specifications are

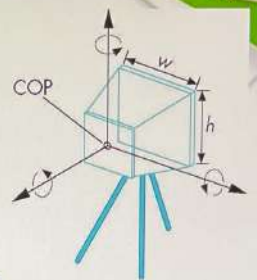
- 1) **position**
- 2) **Orientation (rotation)**
- 3) **Focal length**
- 4) **Film plane**



Viewer or camera

1. **Position:** The camera location usually is given by the position of the center of the lens (**the center of projection**).

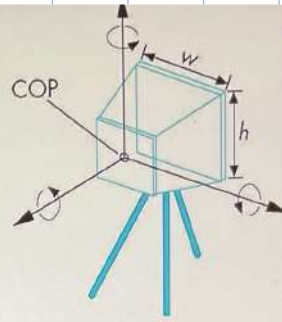
1. **Orientation:** Once the camera is positioned, a camera coordinate system can be placed with its origin at the center of projection. Then the camera can be rotated independently around the three axes of this system.



viewer or camera

3. **Focal length:** The focal length of the lens determines the **size of the image** on the film plane or, equivalently, the portion of the world the camera sees.

3. **Film plane:** The back of the camera has a height and a width. The orientation of the back of the camera can be adjusted independently of the orientation of the lens.



Viewer Specification in OpenGL

1. `gluLookAt()`
2. `gluPerspective()`

1. `gluLookAt(cop_x, cop_y, cop_z, at_x, at_y, at_z, up_x, up_y, up_z);`

- points the camera from COP toward desired point with specified up direction for the camera
- `cop_x, cop_y, cop_z`: eyePosition3D is a XYZ position. This is where you are (your eye is).
- `at_x, at_y, at_z`: center3D is the XYZ position where you want to look at.
- `up_x, up_y, up_z`: upVector3D is a XYZ normalized vector. Often 0.0, 1.0, 0.0

Unit 1

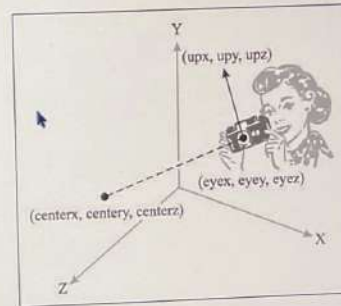
Viewer Specification in OpenGL

Unit 1

```
void gluLookAt (  
    GLdouble eyex, GLdouble eyey, GLdouble eyez,  
    GLdouble centerx, GLdouble centery, GLdouble centerz,  
    GLdouble upx, GLdouble upy, GLdouble upz  
);
```

If you don't call `gluLookAt()`, the OpenGL camera is given some default settings:

- it's located at the origin, (0, 0, 0);
- it looks down the negative Z axis;
- its "up" direction is parallel to the Y axis.

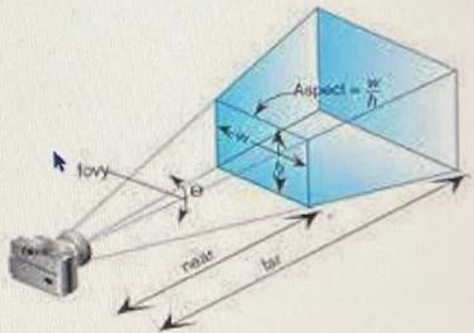


viewer Specification in OpenGL

Unit 1

gluPerspective(field_of_view, aspect ratio, near, far);

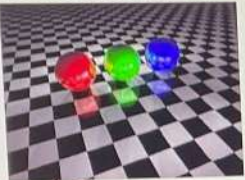
selects a lens for a perspective view and how much of the world the camera should image.



⇒ Light Sources :

Light sources

- Defined by their **location, strength, color and direction**.
- APIs provide a set of functions to specify these parameters.



LIGHT SOURCES

Point Light Spot Light Directional Light

Material Properties

- Material properties are **attributes** of the objects.
- Such properties are usually specified through a series of function calls at the time that each object is defined.



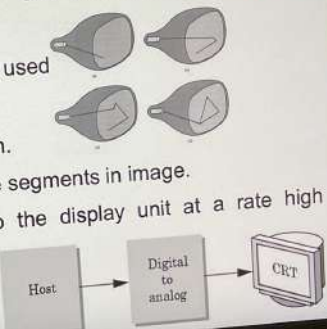
GRAPHICS ARCHITECTURES:

— Categories of Graphics architecture :

- (1) Early Graphics Architectures (EGA)
- (2) Display Processor Architectures (DPA)
- (3) Pipeline Architectures

→ Early Graphics Architecture: (EGA)

- Used computers with standard Von Neumann architecture.
- Had **single processing unit** and could process **only one instruction at a time**.
- Calligraphic CRT** display was used
- Job of host**
 - To run application program.
 - compute end points of line segments in image.
 - To send these points to the display unit at a rate high enough to avoid flicker.



```
graph LR; Host[Host] --> DAC[Digital to analog]; DAC --> CRT[CRT];
```

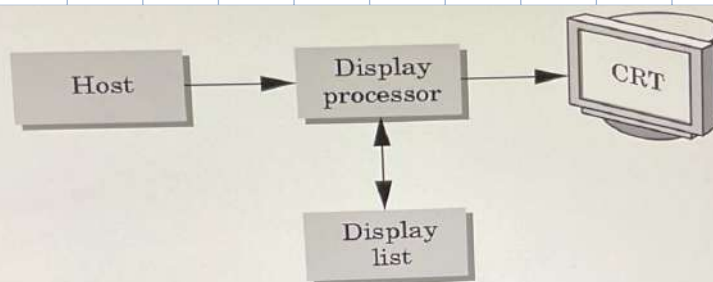
Advantages

- Simple architecture
- Very less expensive

Disadvantages

- Very slow
- Not suitable to produce all types of images. Hence EGA were replaced by DPA.

→ Display Processor Architecture :



- It had the host computer, display processor, display list (display file) and the output device.
- The host computer would assemble the instructions (which can generate the image) once and send it to the display processor.

- The display processor would store these assembled instructions in its own memory called display list.
- The display processor would then execute the display list continuously **independently of the host** at a rate which is sufficient to avoid flicker.

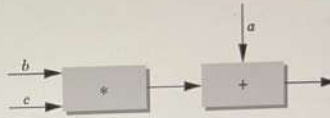
Advantages

- It frees the host computer from repeatedly assembling the instructions.
- Hence the host is free to perform other tasks.

Disadvantages

- Not suitable to produce real time 3D graphics images. Hence it was replaced by pipeline architecture.

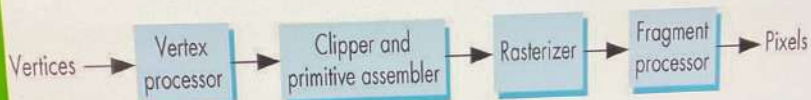
→ Pipeline Architecture:



- Adder and multiplier.
- To calculate $a + (b * c)$
- With many values of a,b,c: multiplier can pass on the results of its calculations to the adder and can start its next multiplication while adder carries out second step of calculation on the first set of data.
- So, the rate at which data flows through the system, the **throughput of the system is doubled.**

- Currently used and the most prominent graphics architecture.
- Makes use of **VLSI circuits** to generate images.
- The major steps involved to form an image

- **Vertex processing**
- **Clipping and primitive assembly**
- **Rasterization**
- **Fragment processing**



a) **Vertex processing**

- First step of Pipeline Architecture.
- **Each vertex is processed independently.**
- Carries out 2 functions
 - **co-ordinate transformation**
 - **computing color of each vertex**
- Successive change of co-ordinate system is represented as a single matrix.

a) **Vertex processing**

- Transformation can be done by multiplying OR concatenating individual matrices to a single matrix.
- After multiple stages of transformation, the geometry is transformed by a projection matrix.
- It also specifies color, properties of the object and light source in the scene.
- The result of this stage is a **set of vertices** that specify the object or group of objects.

b) Clipping and Primitive Assembly

- Second step of Pipeline Architecture.
- Output of vertex processing is given as input to this stage.
- This stage is mainly involved in clipping of the primitive.

• Clipping

Just as a real camera cannot "see" the whole world, the virtual camera can only see part of the world or object space. Objects that are not within this volume are said to be clipped out of the scene.

Clipping

- Within this stage, we must assemble sets of vertices into primitives (line segments or polygons) before clipping takes place.
- The output of this stage is a **set of primitives** whose projections can appear in the image.

c) Rasterization

- Third step of Pipeline Architecture.
- Converts 2D objects into pixels.
- The primitives that are output by the clipper are processed to generate pixels in the frame buffer.
- The output of the rasterizer is a **set of fragments** for each primitive.
- Fragments are "potential pixels" having a location in frame buffer, have Color and depth attributes.

d) Fragment processing

- Final step of Pipeline Architecture.
- Fragments are processed to determine **the color of the corresponding pixel** in the frame buffer.
- Color of a fragment can be altered by **texture mapping**.

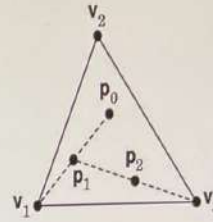
Texture mapping is a method for adding surface texture (a bitmap or raster image), or color to a computer-generated graphic or 3D model. A **texture map** is applied (mapped) to the surface of a shape or polygon.



CONSTRUCTION OF SIERPINSKI GASKET - USING RANDOM NUMBERS :

The construction of sierpinski gasket as follows:

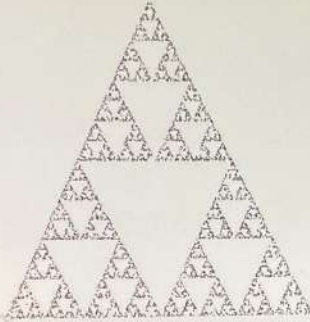
1. Pick up a random point inside the triangle.
2. Select one of the three vertices at random.
3. Find the point halfway between initial point and the randomly selected vertex.
4. **Display this new point** by putting some sort of marker, such as a small circle, at its location.
5. Replace the initial point with this new point.
6. Return to step 2.



Thus, each time a point that is generated, it is displayed on the output device.

In the figure p_0 is the initial point, and p_1 and p_2 are the first two points generated by the algorithm.

random number generation



recursion



Construction of Sierpinski Gasket - using random numbers

Possible form of our program might be:

```
main ( )
{
    initialize_the_system ();
    for (some_number_of_points )
    {
        pt = generate_a_point();
        display_the_point(pt);
    }
    cleanup()
}
```

Construction of Sierpinski Gasket using OpenGL

Unit 1

```
#include <GL/glut.h>
void main (int argc, char **argv) /* the main function */
{
    glutInit(&argc,argv);
    glutDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (500,500);
    glutInitWindowPosition(0,0);
    glutCreateWindow("simple OpenGL example");
    glutDisplayFunc(display);
    myinit( );
    glutMainLoop( ); // Event Processing
}
```

Construction of Sierpinski Gasket using OpenGL

```
typedef GLfloat point2 [2];
/* defines a point as two dimensional array */

void display(void)
{
    /* an arbitrary triangle */
    point2 vertices[3]= { {0.0,0.0}, {250.500}, {500.0,0.0}};
    static point2 p = {75.0, 50.0}; /* set to any initial point */
    int j,k;
    int rand( ); /* standard random number generator */
}
```

Construction of Sierpinski Gasket using OpenGL

Unit 1

```
for (k=0;k<5000;k++)
{
    j = rand ( )%3; /* pick a random vertex from 0, 1, 2 */
    p[0] = (p[0]+ vertices[ j ][0])/2;
    p[1] = (p[1]+ vertices[ j ][1])/2;
    /* display the new point */
    glBegin(GL_POINTS);
    glVertex2fv(p);
    glEnd();
}
glFlush(); /* points are to be displayed as soon as possible */
```

Construction of Sierpinski Gasket using OpenGL

- The function **rand** is a standard random-number generator that produces a new random integer each time that it is called.
- Modulus operator is used to reduce these random integers to the three integers 0, 1, and 2.
- The call to **glFlush** ensures that points are rendered to the screen as soon as possible. If it is left, the program works correctly, but in a busy or networked environment there may be noticeable delay.
- **A complete program is not yet written.**

Basic OpenGL types

- **GLfloat** and **GLint**, are used rather than the C types, such as float and int.
- These types are defined in the header files and usually in the obvious way.
- For example, **#define GLfloat float**
- However, use of the OpenGL types allows additional flexibility for implementations.
- For example, suppose the floats are to be changed to doubles without altering existing application programs.

Basic OpenGL types

- Returning to the vertex function, if the user wants to work in 2-D with integers, then the form **glVertex2i(GLint xi, GLint yi)** is appropriate
- **glVertex3f(GLfloat x, GLfloat y, GLfloat z)** specifies a position in 3-D space using floating-point numbers.
- If an array is used to store the information for a 3-D vertex, **GLfloat vertex[3]** then **glVertex3fv(vertex)** can be used.

Geometric primitives

- Different numbers of vertices are required depending on the object.
- Any number of vertices can be grouped using the **functions glBegin and glEnd**.
- The argument of **glBegin** specifies the geometric type that the vertices define. Hence, a line segment can be specified by

```
glBegin(GL_LINES);
    glVertex2f(x1,y1);
    glVertex2f(x2,y2);
glEnd();
```

APPLICATION AREAS OF COMPUTER GRAPHICS:

- ① Computer-Aided Design (CAD)
- ② Presentation graphics
- ③ Computer Art
- ④ Entertainment
- ⑤ Education and Training
- ⑥ Visualization
- ⑦ Image Processing
- ⑧ GUI

INTRO TO GRAPHICS PROGRAMMING WITH OPEN GL:

- it is not a programming lang like C or C++
- it is a C runtime library which provides some prepackaged functionality
- device independent
- platform independent
- software interface to graphics hardware.
- it uses algos developed and optimized by Silicon Graphics Inc (SGI)
- uses state machine
- procedural, not object-oriented.

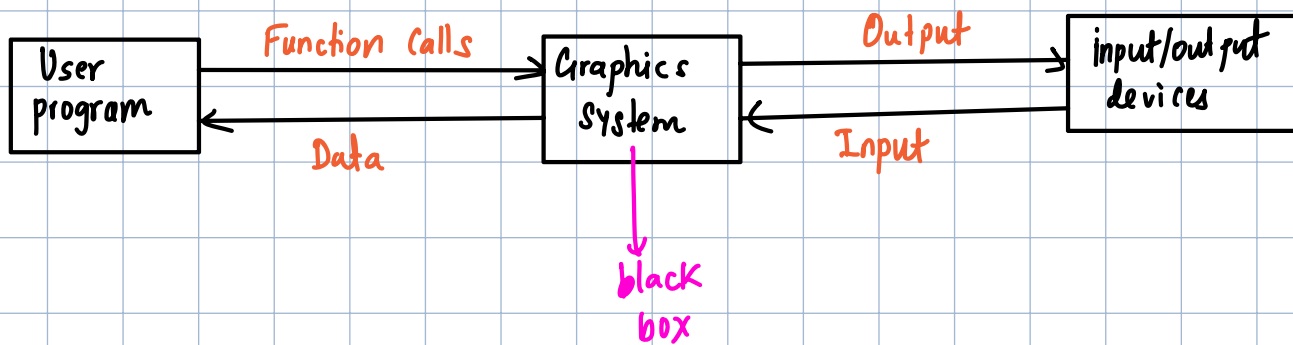
2.2 The OpenGL Library Family

Library	Prefix	Purpose
GL (core library)	gl	Core rendering functions — primitives, transformations, lighting, texturing.
GLU (OpenGL Utility Library)	glu	Higher-level convenience routines built on GL — e.g., setting up perspective projections (<code>gluPerspective</code>), building quadric shapes (spheres/cylinders), mipmap generation.
GLUT / FreeGLUT (OpenGL Utility Toolkit)	glut	Window-system-independent toolkit for creating windows, handling keyboard/mouse events, and running the event loop — used heavily for teaching and quick prototyping.
GLX / WGL / AGL	glX / wgl	Platform-specific "glue" that binds OpenGL to a particular window system (X11, Windows, macOS).

GRAPHIC FUNCTIONS:

→ Basic model of graphics package is a **black box**

system whose properties are described by only its i/p and o/p, nothing is known about its internal workings.



⇒ GRAPHIC FUNCTIONS TYPES:

- ① Primitive Functions
- ② Attribute Functions
- ③ Viewing Functions
- ④ Transformation Functions
- ⑤ Input Functions
- ⑥ Control Functions
- ⑦ Query Functions

① Primitive Functions :

enable the programmer to create some basic primitives such as points, line segment, polygons, text etc

eg:

`glVertex*();`

`glBegin();`

`glEnd();`

Example :

`/* Draw a triangle */`

`glBegin (GL_LINE_LOOP);`

`glVertex3f(-0.3, -0.3, 0.0);`

`glVertex3f(0.0, 0.3, 0.0);`

`glVertex3f(0.3, -0.3, 0.0);`

`glEnd();`

② Attribute Functions :

perform operations ranging from choosing the color, to picking a pattern with which to fill the inside of a polygon, to selecting a type face for the titles on a graph

eg: `glColor3f(1.0, 0.0, 0.0);`

`glClearColor(1.0, 1.0, 1.0);`

③ Viewing Functions :

enable the programmer to specify views such as front view, top view, side view. Also provide zoom in and zoom out facilities

eg: `glViewport();`

`glMatrixMode(GL_PROJECTION);`

`glMatrixMode(GL_MODELVIEW);`

`gluOrtho 2D();`

④ Transformation Functions:

enable the programmer to carryout geometric transformations such as rotation, scaling, translation

eg: `glTranslatef();`
`glRotatef();`

⑤ Input Functions:

enable the programmer to design interactive programs. These functions can accept inputs from mouse, keyboard, data tablets etc.

eg: `glutMouseFunc(mouse);`

⑥ Control functions:

enable the programmer to communicate with the window system, initialize programs, deal with errors that occur during the execution of the program

eg: `glutInitDisplayMode();`
`glutInitWindowSize();`
`glutInitWindowPosition();`

⑦ Query Functions:

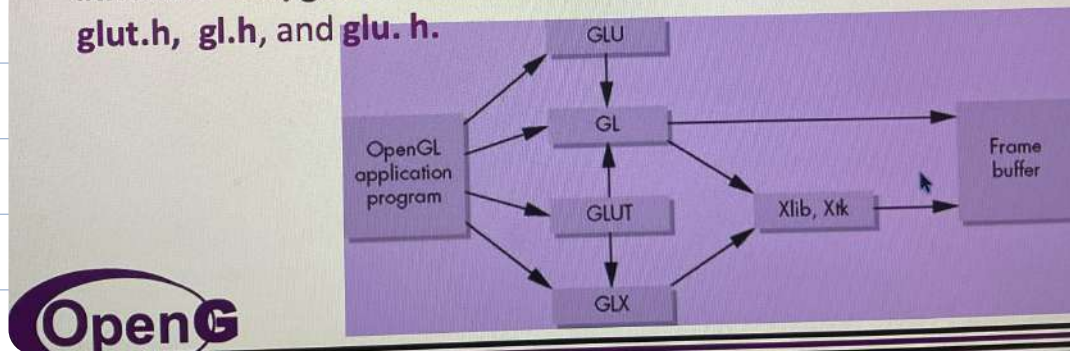
enable programmer to access information such as camera parameters, contents of the frame buffer etc., which the programmer can design device independent programs

eg. `glGetBooleanv()`;
`glGetIntegerv()`;

THE OPENGL INTERFACE:

The OpenGL Interface

- Figure below shows the organization of the libraries for an **X Window system** environment.
- Other libraries are called from the OpenGL libraries, but that the application program does not need to refer to these libraries directly.
- In most implementations, one of the include lines `#include <GL/glut.h>` or `#include <glut.h>` is sufficient to read in `glut.h`, `gl.h`, and `glu.h`.



3.1 Function Naming Conventions

OpenGL function names encode the number and type of arguments they expect. The general pattern is:

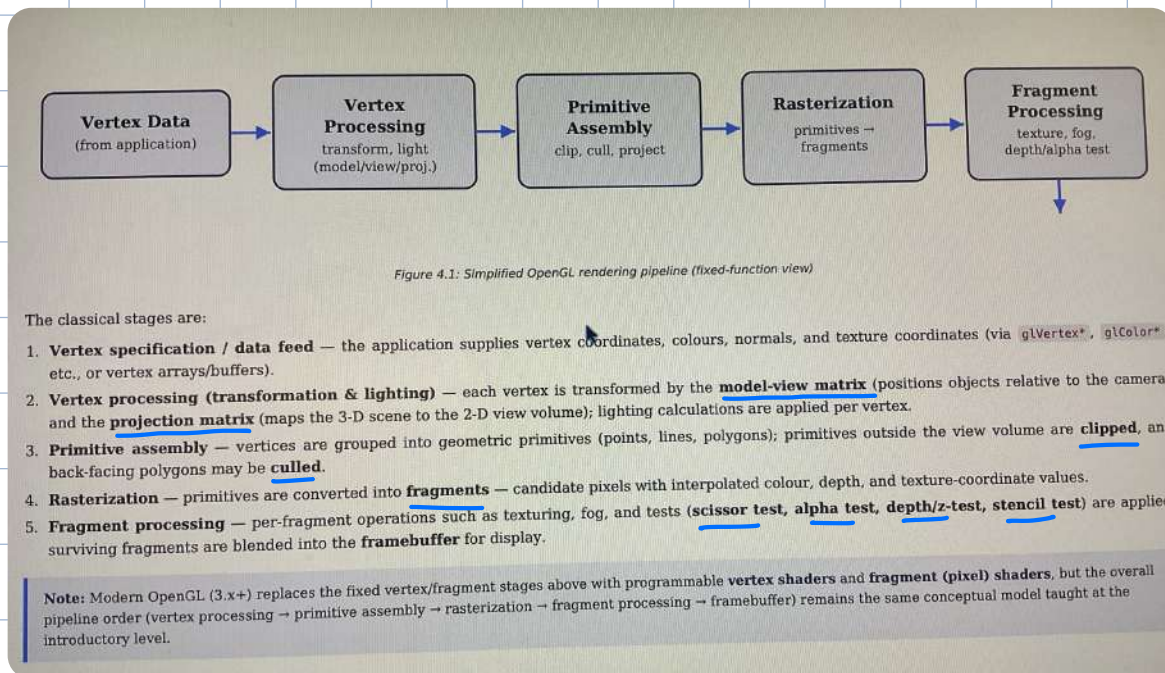
`glName [number-of-args] [type] [v]`

For example, `glVertex3f` and `glVertex2dv` both set a vertex, but differ in dimensionality, data type, and whether the arguments are passed individually or as a vector (array).

Suffix / Letter	Meaning
2, 3, 4	Number of arguments (dimensions) supplied, e.g. (x,y), (x,y,z), (x,y,z,w)
b	GLbyte (8-bit integer)
s	GLshort (16-bit integer)
i	GLint (32-bit integer)
f	GLfloat (32-bit floating point)
d	GLdouble (64-bit floating point)
ub	GLubyte (unsigned byte)
us	GLushort (unsigned short)
ui	GLuint (unsigned int)
v	Arguments are supplied as an array/vector rather than individually

THE GRAPHICS PIPELINE & STATE MACHINES:

The graphics pipeline is the sequence of processing stages that transform a geometric description of a scene (vertices, colours, textures) into a 2-D raster image (pixels) in the frame buffer



⇒ OPEN GL AS A STATE MACHINE:

a state machine is a system whose behaviour depends on internally stored "state" (current settings) as well as its inputs.

→ in Open GL, once you set a piece of state, that state remains in effect for all subsequent drawing operations until it is explicitly changed again

Examples of OpenGL state:

- Current drawing colour (`glColor3f`)
- Current point size / line width (`glPointSize`, `glLineWidth`)
- Whether depth testing, lighting, texturing, or blending are enabled (`glEnable`/`glDisable`)
- The current matrix mode and the contents of the model-view/projection matrices

Because of this state-machine design, the order in which OpenGL calls are issued matters a great deal: e.g., `glColor3f` must be called *before* `glVertex*` for the colour to apply to that vertex.

```
glColor3f(1.0, 0.0, 0.0); // state changed: current colour = red
glBegin(GL_TRIANGLES);
glVertex2f(0.0, 0.0); // drawn in red (uses current state)
glVertex2f(1.0, 0.0);
glColor3f(0.0, 1.0, 0.0); // state changed mid-primitive: colour = green
glVertex2f(0.5, 1.0); // this vertex uses green -> creates a gradient
glEnd();
```

Applications can query the current value of most state variables using `glGetIntegerv`, `glGetFloatv`, `glGetBooleanv`, or `glIsEnabled`, and can save/restore whole groups of state with `glPushAttrib`/`glPopAttrib`.

Exam Tip: A common question is "Explain why OpenGL is called a state machine, with an example." Always mention: (1) the definition, (2) at least 2-3 examples of state variables, and (3) that state persists until changed (give the colour-before-vertex example).

5.2 The glBegin / glEnd Primitive Interface

Geometric primitives in classic (immediate-mode) OpenGL are specified between a `glBegin(mode)` and matching `glEnd()` call. The `mode` argument (a symbolic constant such as `GL_POINTS`, `GL_LINES`, `GL_POLYGON`) tells OpenGL how to interpret the sequence of vertices supplied in between.

```
glBegin(GL_TRIANGLES);
glVertex2f(0.0, 0.0);
glVertex2f(1.0, 0.0);
glVertex2f(0.5, 1.0);
glEnd();
```

Restriction: Between `glBegin` and `glEnd`, only a limited subset of functions may be called — mainly `glVertex*`, `glColor*`, `glNormal*`, `glTexCoord*`, and a few others. Functions like `glBegin` itself, window/viewport calls, or most matrix operations are illegal inside this block.

5.3 The GLUT Callback / Event-Driven Model

OpenGL itself has no notion of a window or of user input — GLUT supplies this by implementing an **event-driven programming model**: the application registers "callback" functions, and GLUT invokes the appropriate one whenever a relevant event occurs.

GLUT Function	Purpose
<code>glutInit(&argc, argv)</code>	Initializes GLUT; must be called before any other GLUT function.
<code>glutInitDisplayMode(mode)</code>	Selects buffering (single/double) and colour mode, e.g. <code>GLUT_DOUBLE</code> <code>GLUT_RGB</code> <code>GLUT_DEPTH</code> .
<code>glutInitWindowSize(w, h)</code>	Sets the initial window size in pixels.
<code>glutInitWindowPosition(x, y)</code>	Sets the initial window's screen position.
<code>glutCreateWindow(title)</code>	Creates the window and its OpenGL rendering context.
<code>glutDisplayFunc(func)</code>	Registers the callback invoked whenever the window must be redrawn.
<code>glutReshapeFunc(func)</code>	Registers the callback invoked when the window is resized.
<code>glutKeyboardFunc(func)</code>	Registers the callback for standard keyboard key presses.
<code>glutMouseFunc(func)</code>	Registers the callback for mouse button events.
<code>glutIdleFunc(func)</code>	Registers a callback executed continuously when no other events are pending (used for animation).
<code>glutMainLoop()</code>	Enters the GLUT event-processing loop; never returns.

OPENGL PRIMITIVES & ATTRIBUTES:

⇒ PRIMITIVES:

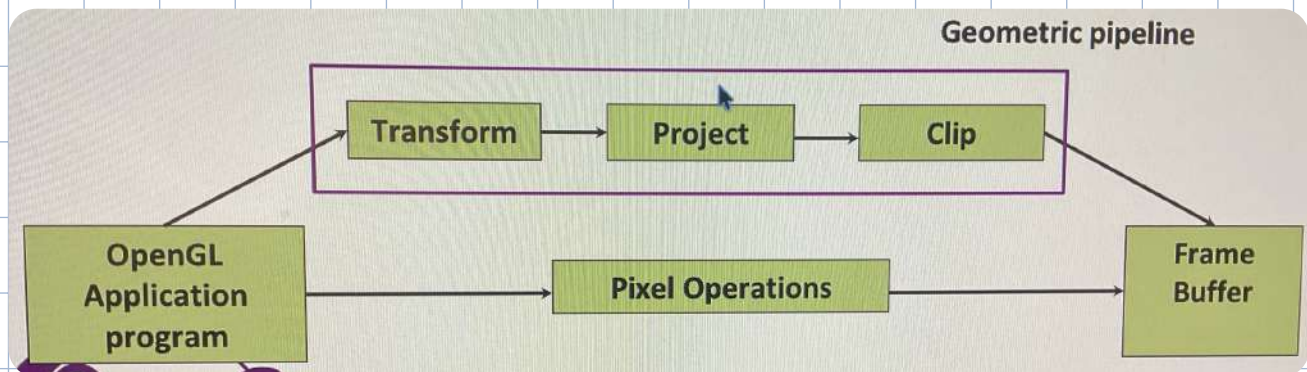
→ OpenGL supports the following primitives:

① Geometric primitives:

- points
- line segments
- polygons
- curves
- surfaces

② Image (or) Raster Primitives

- array of pixels



→ Geometric Primitives:

`glBegin(type);` → specifies how OpenGL assembles the vertices to define geometric objects
`glVertex*(...);`
:
`glVertex*(...);`

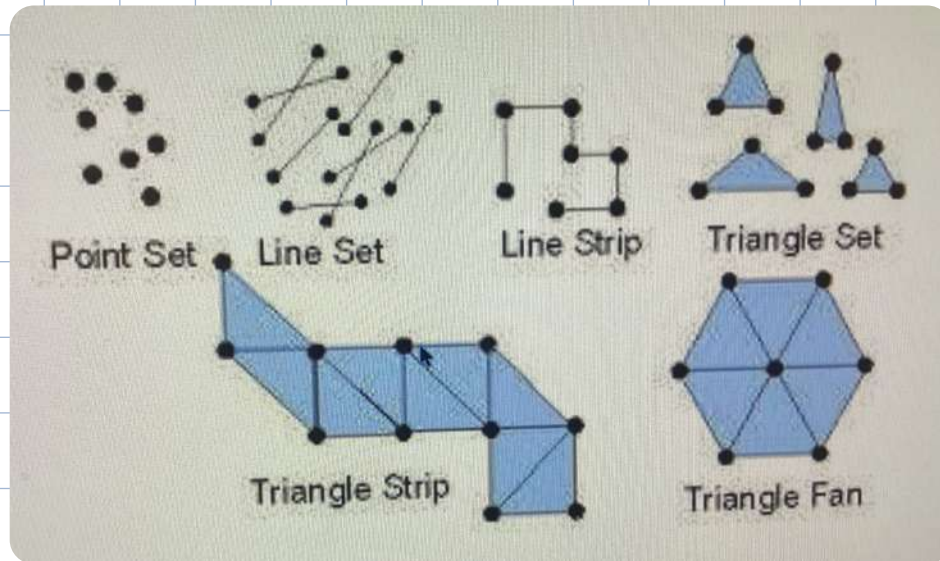
`glEnd();`

1) Points: (GL_POINTS)

2) Line segment: (GL_LINES)

3) Polyline: (GL_LINE_STRIP, GL_LINE_LOOP)

4) Polygons : (GL_TRIANGLES),
(GL_QUADS),
(GL_POLYGON)



- In OpenGL, internal representation for both 2-D and 3-D points is same. Hence a 3-D point is represented by a triplet:

$$P = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

- **Vertex (rather than point):**
 - A vertex is a location in space (in Computer Graphics a 2-D, 3-D, 4-D spaces are used).
 - Vertices are used to define the atomic geometric objects that are recognized by graphics system.

⇒ ATTRIBUTES :

↳ properties that control how a primitive appears when rendered — its color, thickness, style and fill pattern.
a primitive by itself only defines geometry ; attributes defines its appearance

Because OpenGL is a state machine (Section 4.2), attributes are set *before* the primitive is specified, and remain in effect for every subsequent primitive until changed. Broad attribute categories are:

- **Colour attributes** — `glColor*` (Section 9).
- **Point attributes** — `size (glPointSize)`.
- **Line attributes** — `width (glLineWidth)` and stipple pattern (`glLineStipple`).
- **Polygon (area-fill) attributes** — fill style, stipple pattern, front/back-face designation, and edge flags (`glPolygonMode`, `glPolygonStipple`, `glFrontFace`).

Detailed function signatures and examples for each category are given in Section 8.

⇒ VERTEX FUNCTION :

- A vertex is a position in space.
- Space can be 2-D or 3-D.
- Vertex is used to define geometric primitives.

Example

Two vertices define a line segment. In OpenGL it is represented as

`glVertex*()` where * can be nt OR ntv

- n no. of dimensions (2, 3, 4)
- t data types (int, float, double)
- v specifies that variables are specified through a pointer to an array.

Basic OpenGL data types include

GLfloat, GLint, GLdouble

Vertex in 2D can be represented as

`glVertex2i(GLint xi, GLint yi)`

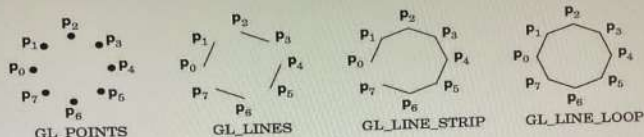
In OpenGL, primitives are defined between **glBegin** and **glEnd**

glBegin – The argument specifies the geometric type of vertices.

Example :

```
glBegin(GL_LINES);
glVertex3f(x1, y1, z1);
glVertex3f(x2, y2, z2);
glEnd();
```

x1, y1, z1 x2, y2, z2



GL_POINTS : Each vertex is displayed at a size of at least one pixel.

GL_LINES : Successive pairs of vertices would be connected as a line.

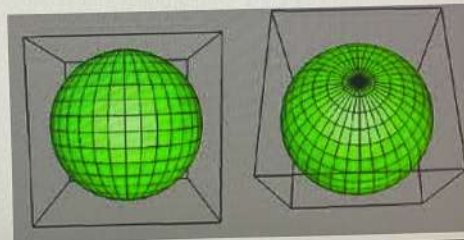
GL_LINE_STRIP : Connects the successive vertices using line segments. However the final vertex would not be connected to the initial vertex.

GL_LINE_LOOP : Connects the successive vertices using line segments to form a closed path.

POLYGON BASICS:

→ polygon is a filled line-loop structure

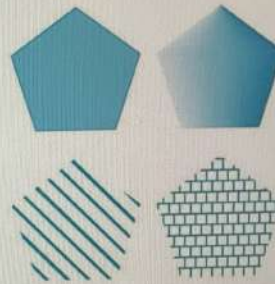
- Polygons play a special role in computer graphics because they can be displayed rapidly and they can be used to approximate arbitrary or curved surfaces.
- The performance of graphics systems is measured by the number of polygons per second that can be displayed.



OpenG

Ways of displaying polygons

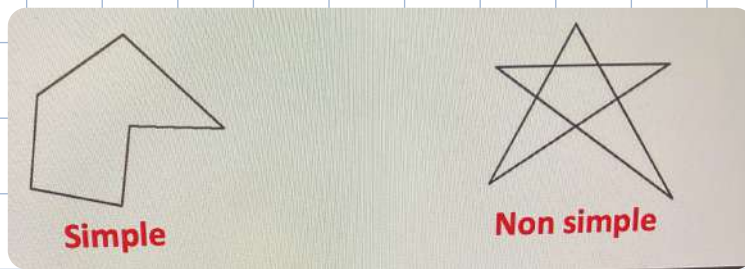
- Only its edges be displayed.
- Its interior be filled with a solid color, or a pattern.
- Either display or not display the edges.
- Outer edges of a polygon can be defined easily by an ordered list of vertices.
- But if the interior is not well defined, then the polygon may be rendered incorrectly.



⇒ PROPERTIES OF A POLYGON:

① Simple:

- ° a 2-D polygon in which no pair of edges cross each other
- ° they will have well-defined interiors



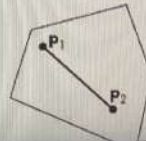
② Convex:

• a convex polygon is a multi-angled shape with straight sides in which all of the angles are $< 180^\circ$.

eg: pentagon

• a line drawn through a convex polygon will intersect the sides of the polygon exactly twice

- All triangles and regular polygons such as squares, parallelograms, trapezoids, and rectangles are convex because each of their interior angles is less than 180° .
- All the points between p_1 and p_2 will be inside the polygon **(on the surface of the polygon)**.



③ Flat/ Planar:

• the vertices lie in the same plane.

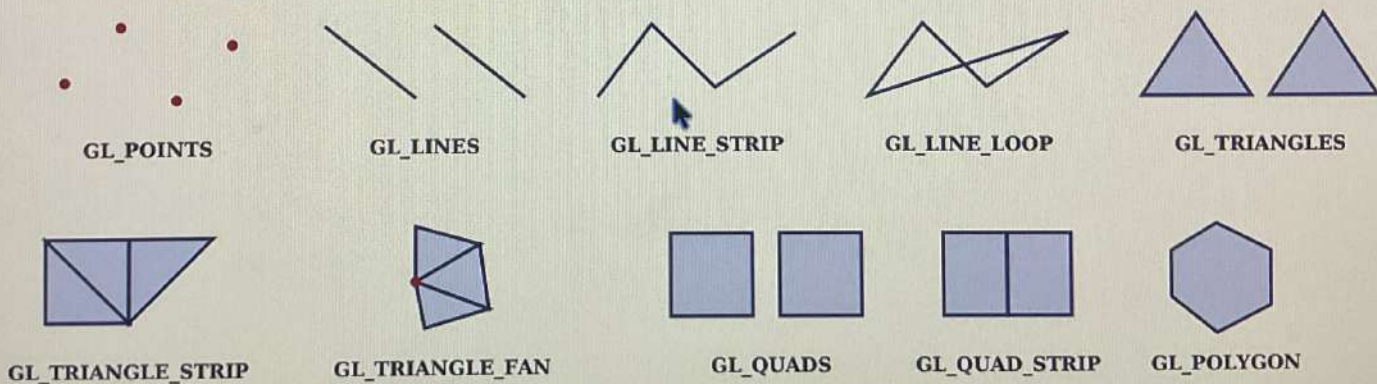
• in 3D, all vertices that define polygon need to lie in the same plane.

7.2 The Ten Primitive Types (glBegin Modes)

The symbolic constant passed to `glBegin()` determines how the list of vertices supplied afterward is interpreted:

Constant	Category	Interpretation
<code>GL_POINTS</code>	Point	Each vertex is drawn as an individual point.
<code>GL_LINES</code>	Line	Each independent pair of vertices (v_0-v_1 , v_2-v_3 , ...) forms a separate line segment.
<code>GL_LINE_STRIP</code>	Line	Consecutive vertices are connected: v_0-v_1 , v_1-v_2 , v_2-v_3 , ... (an open polyline).
<code>GL_LINE_LOOP</code>	Line	Same as <code>LINE_STRIP</code> but an extra segment closes the loop from the last vertex back to the first.
<code>GL_TRIANGLES</code>	Polygon	Every independent group of 3 vertices forms a separate triangle.
<code>GL_TRIANGLE_STRIP</code>	Polygon	Each additional vertex, combined with the previous two, forms a new triangle (efficient strip of connected triangles).
<code>GL_TRIANGLE_FAN</code>	Polygon	All triangles share the first vertex specified as a common "hub"; each new vertex forms a triangle with the hub and the previous vertex.
<code>GL_QUADS</code>	Polygon	Every independent group of 4 vertices forms a separate quadrilateral.
<code>GL_QUAD_STRIP</code>	Polygon	Forms a strip of connected quadrilaterals from successive pairs of vertices.
<code>GL_POLYGON</code>	Polygon	All specified vertices form a single simple convex polygon (an n-gon).

Note: `GL_QUADS`, `GL_QUAD_STRIP`, and `GL_POLYGON` were removed from modern "core-profile" OpenGL (3.1+) because GPUs natively rasterize only points, lines, and triangles — but they remain part of the classic/compatibility API taught in this course and are fully valid in the textbook (Hearn & Baker) examples.

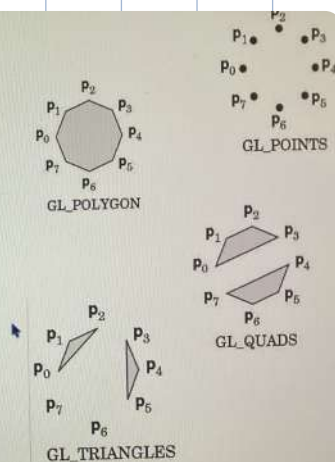


Polygon Types in OpenGL

GL_POLYGON : Connects the successive vertices using line segments to form a closed path. The interior is filled according to the state of the relevant attributes.

GL_QUADS : Special case of polygon where successive group of 4 vertices are interpreted as quadrilaterals.

GL_TRIANGLES : Special case of polygon where successive group of three vertices would be interpreted as a triangle.

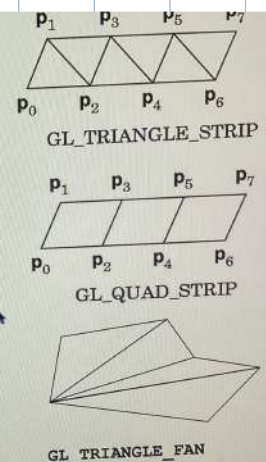


Polygon Types in OpenGL

GL_TRIANGLE_STRIP : Each additional vertex is combined with the previous two vertices to define a new triangle.

GL_QUAD_STRIP : Two new vertices are combined with the previous two vertices to design a new quadrilateral.

GL_TRIANGLE_FAN : Based on one fixed point. The next two points determine the first triangle. The subsequent triangles are formed from one new point, the previous point and the first (fixed) point.



Curved Primitives

- Curved primitives such as sphere, oval, parabola can be created using **tessellation**.
- **Tessellation** is a process of creating curved primitives by using a mesh of polygon of n sides.
- **Example** : Approximating a Sphere

⇒ APPROXIMATING A SPHERE:

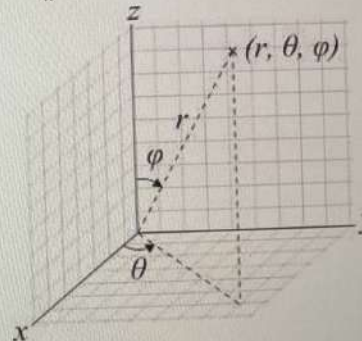
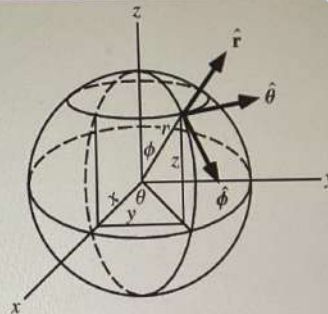
Approximating a Sphere

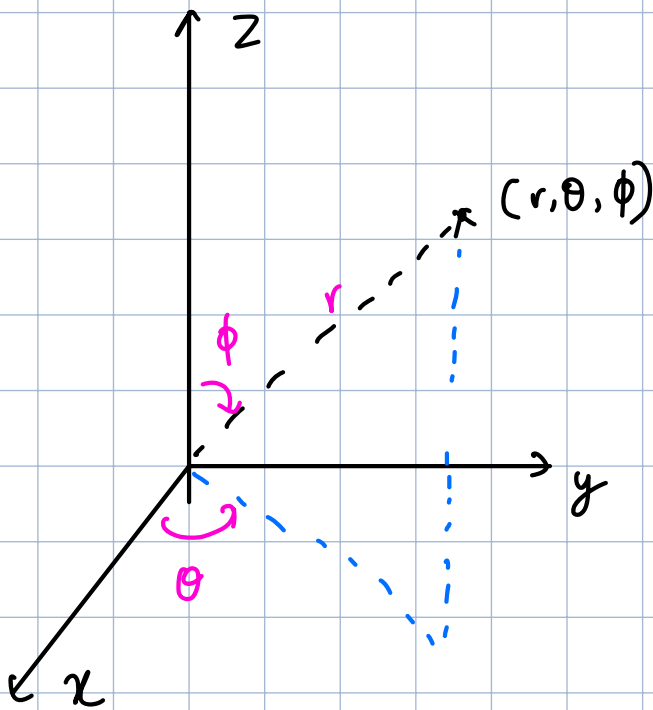
- Many curved surfaces can be approximated using fans and strips.
- E.g. To approximate to a sphere, a set of polygons defined by lines of longitude and latitude as shown can be used.
- Either quad strips or triangle strips can be used for the purpose.
- Consider a unit sphere ($r=1$). It can be described by the following three equations:

$$x(\theta, \phi) = \sin\theta \cos\phi$$

$$y(\theta, \phi) = \cos\theta \cos\phi$$

$$z(\theta, \phi) = \sin\phi$$

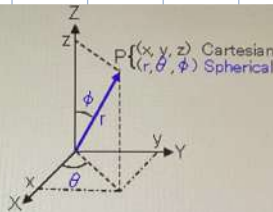
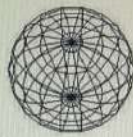




$$\begin{aligned}x(\theta, \phi) &= \sin\theta \cos\phi \\y(\theta, \phi) &= \cos\theta \cos\phi \\z(\theta, \phi) &= \sin\phi\end{aligned}$$

Approximating a Sphere

$$\begin{aligned}x(\theta, \phi) &= \sin\theta \cos\phi \\y(\theta, \phi) &= \cos\theta \cos\phi \\z(\theta, \phi) &= \sin\phi\end{aligned}$$



- If we **fix** ϕ and draw curves, as we change θ , we get circles of **constant latitude**.
- If we **fix** θ and draw curves, as we change ϕ , we get circles of **constant longitude**.
- By generating points at fixed increments of θ and ϕ , we can define quadrilaterals.
- ϕ must be ranged between -80 to +80.
- θ must be ranged between -180 and 180.

★ Degrees must be converted to radians for standard trigonometric functions

CODE:

float c = 3.142 / 180; //converting to radians

// to get longitudes

for (phi = -80; phi <= 80; phi += 20) {

 phir = c * phi;

 phir20 = c * (phi + 20); // for 2nd point

```
glBegin(GL_QUAD_STRIP);
```

```
// to get latitude
```

```
for (theta = -180.0 ; theta <= 180.0 ; theta += 20.0) {
```

```
    thetar = c * theta;
```

```
    x = sin(thetar) * cos(phir);
```

```
    y = cos(thetar) * cos(phir);
```

```
    z = sin(phir);
```

```
    glVertex3d(x, y, z); // first point
```

```
    x = sin(thetar) * cos(phir20);
```

```
    y = cos(thetar) * cos(phir20);
```

```
    z = sin(phir20);
```

```
    glVertex3d(x, y, z); // 2nd point
```

```
}
```

```
glEnd();
```

```
}
```

```
// 1st pole - use triangle fans
```

```
glBegin(GL_TRIANGLE_FAN);
```

```
glVertex3d(0.0, 0.0, 1.0); // top pole
```

```
c80 = c * 80.0
```

```
z = sin(c80);
```

```
for (theta = -180.0 ; theta <= 180.0 ; theta += 20.0) {
```

```
    thetar = c * theta;
```

```
    x = sin(thetar) * cos(c80);
```

```
    y = cos(thetar) * cos(c80);
```

```

    glVertex3d(x, y, z);
}
glEnd();

```

// 2nd pole - using triangle fans

```

glBegin(GL_TRIANGLE_FAN)
glVertex3d(0.0, 0.0, -1.0); // bottom pole
z = -sin(c80);
for (theta = -180.0; theta <= 180.0; theta += 20.0) {
    theta *= c80;
    x = sin(theta) * cos(c80);
    y = cos(theta) * cos(c80);
    glVertex3d(x, y, z);
}
glEnd();

```

⇒ TEXT:

→ type, style, size, colors

→ fonts - Times, Computer Modern or Helvetica

→ Types:

(1) Stroke text:

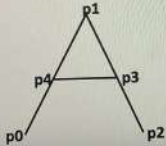
```
void glutStrokeCharacter(void *font, int character)
```

(2) Raster text:

```
void glutBitmapCharacter(void *font, int character)
```

Stroke Text

- It is constructed **similar to geometric objects**.
- Vertex is used to define the line segment or curves that outline each character.
- The characters defined by closed boundaries can be filled.
- Stroke fonts behave like any 3D object, i.e. the characters can be rotated, scaled, and translated.



Stroke Text

Advantages

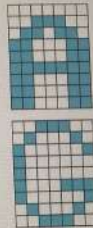
- It can be translated, rotated, scaled etc.
- It can be viewed using different views.

Disadvantages

- Consumes lot of memory.
- Consumes lot of processing time.

Raster text

- It is simple and fast.
- Characters are defined as rectangles of bits called **bit blocks**.
- Each block defines a single character by the pattern of 0 and 1 bits in the block.
- A raster character can be placed in the frame buffer rapidly by a **bit-block-transfer operation**, which moves the block of bits using a single instruction.



Raster text

- OpenGL allows the application program to use instructions that allow direct manipulation of the contents of the frame buffer.
- Size of raster characters can be increased only by replicating or duplicating pixels, a process that gives larger characters a blocky appearance.



Advantages

- Can be placed in the frame buffer rapidly by a bit block transfer.
- Simple and fast.

Disadvantages

- Rotation, scaling etc cannot be applied.
- Raster characters often are stored in read-only memory (ROM) in the hardware and hence, a particular font might be of limited portability.

⇒ ATTRIBUTES:

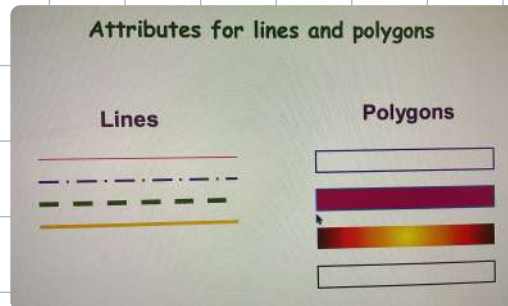
→ property of a geometric primitive that refers to its color, thickness, pattern, edged, non-edged etc.

→ each geometric type has a set of attributes such as

(1) point — color + thickness

(2) line — color + thickness + pattern

(3) Polygon — color + thickness + pattern + display only edge + display without edges



⇒ COLOR:

→ most important att of any geometric primitive

→ a visible color can be characterized by function $C(x)$ that occupies wavelengths from about 350 nm to 780 nm.

→ the human visual system has 3 types of cones responsible for color vision

↓
tristimulus values



3 color theory

* if 2 colors produce same tristimulus values, then they are visually indistinguishable

→ Tristimulus values:

→ any color which can be produced by 3 primary colors —
Blue, Green, Red

//

$$C = rR + gG + bB$$

• R, G, B

↪ unit values

• r, g, b

↪ magnitudes or relative intensities of those primaries

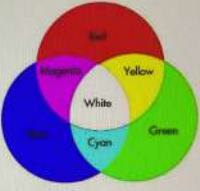
↪ tristimulus values

→ Color Modes:

① Additive Color:

Additive Color

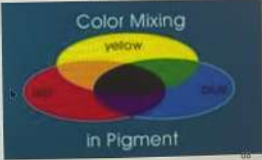
- Primary colors used (**RGB**)
 - Red
 - Green
 - Blue.
- CRT is an example of **additive color** where primary colors are added together to give the perceived color.




② Subtractive Color:

Subtractive Color

- It suits for processes such as **commercial printing and painting** as white surface is used as background.
- The primaries are usually the complementary colors (**CMY**)
 - Cyan
 - Magenta
 - Yellow

Differences between Additive color and Subtractive color

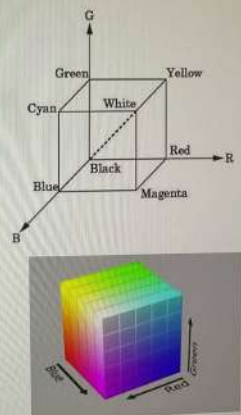
Additive Color	Subtractive Color
Used in CRTs.	Used for commercial printing and painting.
Primary colors are Red, Green, Blue (RGB).	Primary colors are Cyan, Magenta, Yellow (CMY).
Primaries add light to an initially black display to give desired color.	Primaries subtract color from an initially white display to give desired color.



- We can view color as a point in a color solid
- The vertices of a cube correspond to:

black(no primaries on)
red, green, blue(one primary fully on)
cyan(green and blue fully on)
magenta(red and blue fully on)
yellow(red and green fully on)
white(all primaries fully on)

- All colors along principal diagonal line have equal tristimulus values and appear as shades of gray.



⇒ COLOR MODELS:

→ there are 2 color models:

- ① RGB color model
- ② Index color model

① RGB Color Model:

→ in additive RGB sys, there are separate buffers for red, green & blue.

→ each pixel has separate R, G, B components that corresponds to location in memory.

$$\begin{array}{ccccc} 8 \text{ bits} & + & 8 \text{ bits} & + & 8 \text{ bits} & = & 24 \text{ Bits} \\ (R) & & (G) & & (B) & & \end{array}$$



2^{24} diff colors

in OpenGL:

```
void glColor3f(GLfloat R, GLfloat G, GLfloat B);
```

↓
(0.0 → 1.0)

default value ⇒ 0.0

→ RGBA model:

↪ alpha

↓
for creating fog effects (combining the image)

$\begin{bmatrix} 0.0 \rightarrow \text{transparent} \\ 1.0 \rightarrow \text{opaque} \end{bmatrix}$

`glClearColor (1.0, 1.0, 1.0, 1.0)`

↳ will set the background color of window to white and opaque.

② Indexed Color:

→ more commonly used.

→ it is used when frame buffer has limited depth (no. of bits / pixel)

Example :

If the frame buffer has 1280 x 1024 array of pixels but with each pixel consisting of only 8 bits, then color index model is used.

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
⋮	0	$2^m - 1$	0
⋮	⋮	⋮	⋮
$2^k - 1$	⋮	⋮	⋮

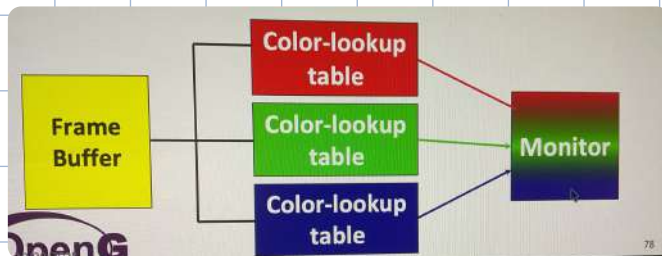
$\xrightarrow{m \text{ bits}} \quad \xrightarrow{m \text{ bits}} \quad \xrightarrow{m \text{ bits}}$

Indexed Color

- In the model, suppose there are 'k' bits per pixel, then each pixel value is an integer between 0 to $2^k - 1$.
- We can display colors with a precision of m bits.
- We can choose from 2^m reds, 2^m greens and 2^m blues.
- Hence we can produce any of 2^{3m} colors on the display but the frame buffer can specify only 2^k of them.
- `glIndexi(element);`
- `glutSetColor(int color, GLfloat red, GLfloat green, GLfloat blue);`

Input	Red	Green	Blue
0	0	0	0
1	$2^m - 1$	0	0
⋮	0	$2^m - 1$	0
⋮	⋮	⋮	⋮
$2^k - 1$	⋮	⋮	⋮

$\xrightarrow{m \text{ bits}} \quad \xrightarrow{m \text{ bits}} \quad \xrightarrow{m \text{ bits}}$



Indexed Color

Advantages

- We can operate with such systems where the **number of bits per pixel is less**.
- The programmer can change the lookup table according to his requirements and hence has the flexibility to enjoy the usage of all possible colors.

Disadvantages

- All possible colors would not be available for usage simultaneously.

→ Setting of Color Attributes:

① `glClearColor (1.0, 1.0, 1.0, 1.0) ;`

background color is set to white

② `glColor3f (0.0, 1.0, 0.0)`

rendering color for the points is set to green.

③ `glPointSize (2.0);`

the size of rendered points is set to 2 pixels wide

CONTROL FUNCTIONS :

→ Control Functions enable the programmer to communicate with the window system, initialize the program etc.

(1) `glutInit ( int *argc , char **argv)`

(2) `glutCreateWindow ( char *title)`

(3) `glutInitDisplayMode ( )`

(4) `glutInitWindowSize ( )`

(5) `glutInitWindowPosition ( )`

(6) `glutDisplayFunc ( )`

(7) `glutMainloop ( )`

Control Functions

1. `glutInit(int *argcp, char **argv)`
 - It initiates an interaction between the windowing system and OpenGL. It is used before opening a window in the program.
 - The two arguments allow the user to pass command-line arguments, as in the standard C main function, and are usually the same as in main.
2. `glutCreateWindow(char *title)`
 - It opens an OpenGL window.
 - The title string provides title at the top to the window displayed.



85

Control Functions

3. `glutInitDisplayMode()`
 - Used to initialize the display window created, by specifying color model, buffering used etc.
4. `glutInitWindowSize()`
 - Used to specify the size of the window to be created.
5. `glutInitWindowPosition()`
 - Used to specify the position of the created window on the monitor with respect to the top left corner.



86

Control Functions

6. `glutDisplayFunc()`
 - Used to specify the display call back.
 - This function executes when the window is created for the first time.
 - It is also called when the window is moved from one location on the screen to another.
7. `glutMainLoop()`
 - Causes the program to begin an event processing loop.
 - If there are no events to process, then the program would enter the wait state with the output on the screen.
 - It is similar to `getch()` in C program.



87

VIEWING:

→ viewing is a type of graphics functions which enables the programmer to specify the view facilities such as top view, front view, left side view, right side view, zoom in, zoom out.

→ determines the appearance of the obj on the o/p device.

* if the programmer does not specify a view explicitly, default one (orthogonal view) is considered

→ Viewing can be:

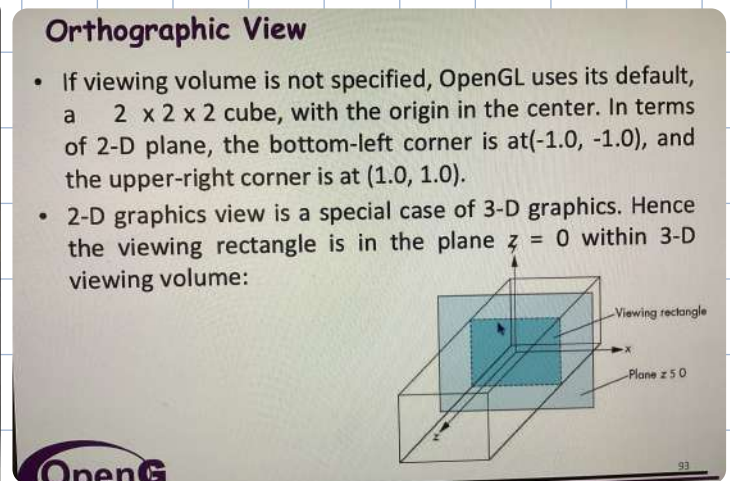
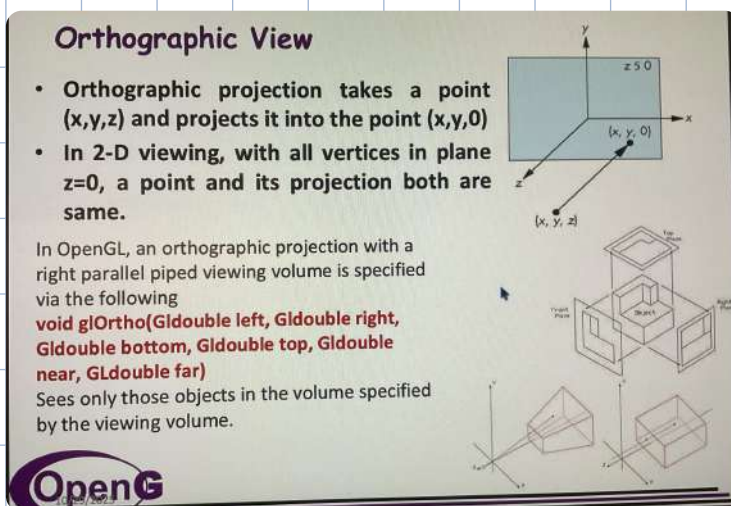
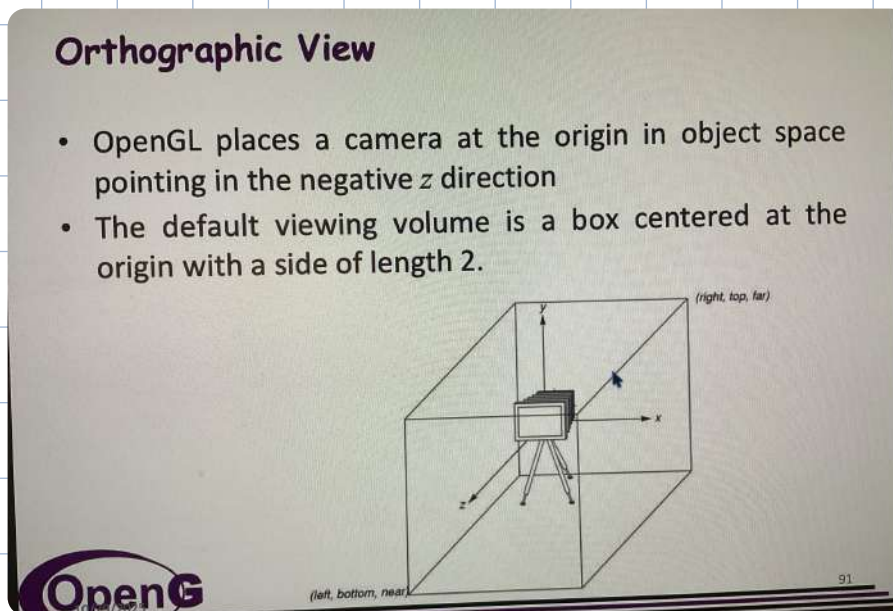
(1) orthographic view

(2) 2D view

(3) Matrix Mode.

① Orthographic View:

- simplest and OpenGL's default view is orthographic
- can be obtained in the synthetic camera model by placing the camera at an ∞ far distance from the obj
- the projections (lines) are $\perp$ (orthogonal) to the projection plane



② 2D-Viewing:

→ is based on taking a rectangular area of 2D world and transferring its contents to the display.

VIEWING RECTANGLE

or

CLIPPING RECTANGLE

→ objects inside the rectangle are in the image, objs outside are clipped out and are not displayed

→ objs that straddle the edges of the rectangle are partially visible in the image

`void glutOrtho2D (GLdouble left, GLdouble right, GLdouble bottom, GLdouble top);`



Same as `glutOrtho`, but `near = -1.0`
`far = 1.0`

③ Matrix Modes:

Matrix modes

- Pipeline graphics systems have an architecture that depends on multiplying together, or concatenating, a number of transformation matrices to achieve the desired image of a primitive.
- The two most important matrices are
 - Model-view
 - Projection matrices
- At any time, the state includes values for both of these matrices, which start off set to identity matrices.
- The usual sequence is to modify the initial identity matrix, by applying a sequence of transformations.

Matrix modes

- Setting the matrix mode selects the matrix to which the operations apply. It is a variable that is set to one type of matrix and is also part of the state.
- The default matrix mode is to have operations apply to the model-view matrix.
- So to alter the projection matrix, modes must be switched. The following sequence is common for setting a 2-D viewing rectangle:
- `glMatrixMode(GL_PROJECTION);`
- `glLoadIdentity();`
- `gluOrtho2D(0.0, 500.0, 0.0, 500.0);`
- `glMatrixMode(GL_MODELVIEW);`

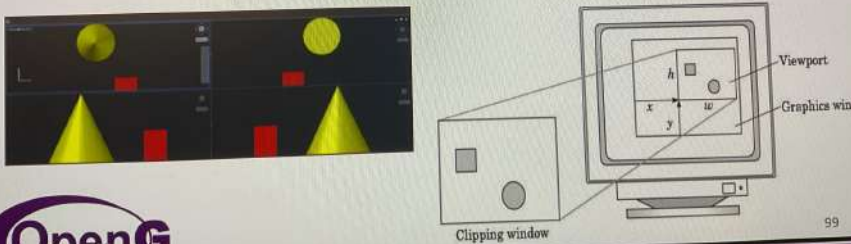
Matrix modes

- `glMatrixMode(GL_PROJECTION);`
- `glLoadIdentity();`
- `gluOrtho2D(0.0, 500.0, 0.0, 500.0);`
- `glMatrixMode(GL_MODELVIEW);`
- This sequence defines a 500 x 500 viewing rectangle with the lower-left corner of the rectangle at the origin of the 2-D system.
- It then switches the matrix mode back to model-view mode.
- In complex programs, it is always better to return to a given matrix mode, to avoid possible confusion over the current mode. E.g. In this case mode returned to model-view mode.

ACTION RATIO & VIEWPORTS:

Aspect Ratio and Viewports

- **Aspect ratio** : ratio of the rectangle's width to its height.
- **Viewport** : Rectangular area of the display window.
- By default, it is the entire window, but can be set to any smaller size in pixels through function.



Aspect Ratio and Viewports

`void glViewport(GLint x, GLint y, GLsizei w, GLsizei h)`

(x,y)

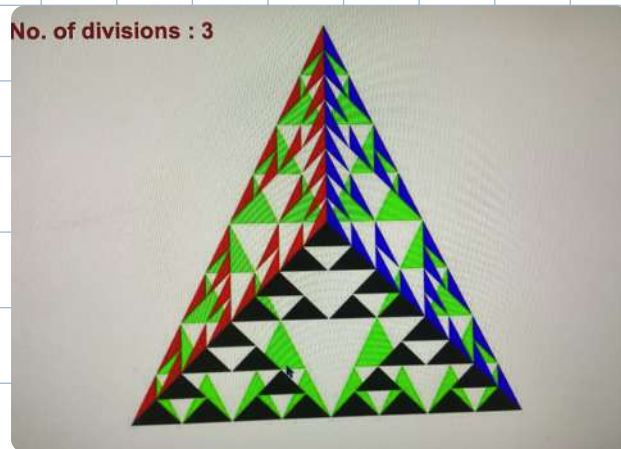
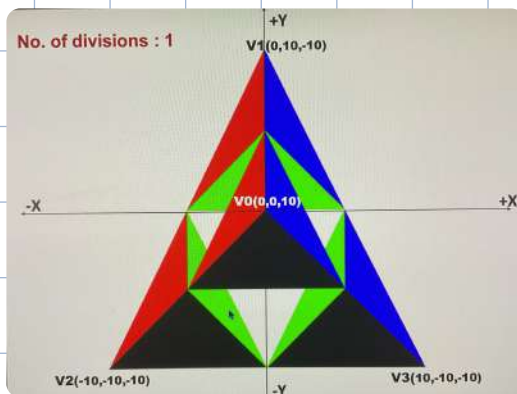
- Specify the lower left corner of the viewport rectangle, in pixels.
- The initial value is (0,0).

w and h (width and height)

- Specify the width and height of the viewport.
- When a GL context is first attached to a window, width and height are set to the dimensions of that window.

3D Sierpinski Gasket:

- a fractal which can be constructed by a recursive procedure;
- at each step a triangle is divided into 4 new As
- the central A is removed and each of the other 3 treated as the original was, and so on, creating an ∞ regression in a finite space.



CODE:

```
#include <stdio.h>
#include <GL/glut.h>

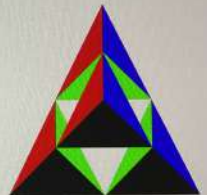
typedef float point[3];

/* initial tetrahedron */
point v[4] = { {0.0, 0.0, 10.0}, {0.0, 10.0, -10.0},
               {-10.0, -10.0, -10.0}, {10.0, -10.0, -10.0} };
int n;

/* display one triangle */
void triangle( point a, point b, point c)
{
    glBegin(GL_POLYGON);
        glVertex3fv(a);
        glVertex3fv(b);
        glVertex3fv(c);
    glEnd();
}
```

```
/* triangle subdivision */
void divide_triangle(point a, point b, point c, int m)
{
    point p1, p2, p3;
    int i;
    if(m>0)
    {
        for(i=0; i<3; i++) p1[i]=(a[i]+b[i])/2;
        for(i=0; i<3; i++) p2[i]=(a[i]+c[i])/2;
        for(i=0; i<3; i++) p3[i]=(b[i]+c[i])/2;

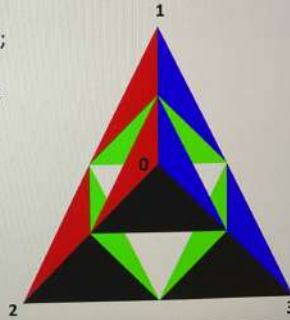
        divide_triangle(a, p1, p2, m-1);
        divide_triangle(c, p2, p3, m-1);
        divide_triangle(b, p3, p1, m-1);
    }
    else
        triangle(a,b,c); /* draw triangle at end of recursion */
}
```



/ Apply triangle subdivision to faces of tetrahedron */*

void tetrahedron(int m)

```
{
    glColor3f(1.0,0.0,0.0); //anti clockwise-visible
    divide_triangle(v[0], v[1], v[2], m);
    glColor3f(0.0,1.0,0.0); //clockwise-not visible surface
    divide_triangle(v[3], v[2], v[1], m);
    glColor3f(0.0,0.0,1.0);
    divide_triangle(v[0], v[3], v[1], m);
    glColor3f(0.0,0.0,0.0);
    divide_triangle(v[0], v[2], v[3], m);
}
```



void display()

```
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    // to reset the buffer and for hidden surface removal
    glLoadIdentity(); // loading identity matrix
    tetrahedron(n);
    glFlush();
}
```

void myReshape(int w, int h) // zoom in and zoom out

```
{
    glViewport(0, 0, w, h); // to set the viewport according to changed w and h
    glMatrixMode(GL_PROJECTION);
    // matrix mode – defines CTM, GL_PROJECTION to view in ortho
    glLoadIdentity(); // identity matrix resets the matrix to its default state
    if (w <= h)
        glOrtho(-12.0, 12.0, -12.0 * (GLfloat) h / (GLfloat) w, 12.0 * (GLfloat) h / (GLfloat) w, -12.0, 12.0);
    else
        glOrtho(-12.0 * (GLfloat) w / (GLfloat) h, 12.0 * (GLfloat) w / (GLfloat) h, -12.0, 12.0, -12.0, 12.0);
    glMatrixMode(GL_MODELVIEW); // transformations done in model view
    glutPostRedisplay();
}
```

void main(int argc, char **argv)

```
{
    printf(" No. of Divisions ? ");
    scanf("%d",&n);
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
    /* This is to view the behind triangle */
    glutInitWindowSize(500, 500);
    glutCreateWindow("3D Gasket");
    glClearColor (1.0, 1.0, 1.0, 1.0);
    glutReshapeFunc(myReshape);
    glutDisplayFunc(display);
    glEnable(GL_DEPTH_TEST); // enable the hidden surface
    glutMainLoop();
}
```

INPUT & INTERACTION

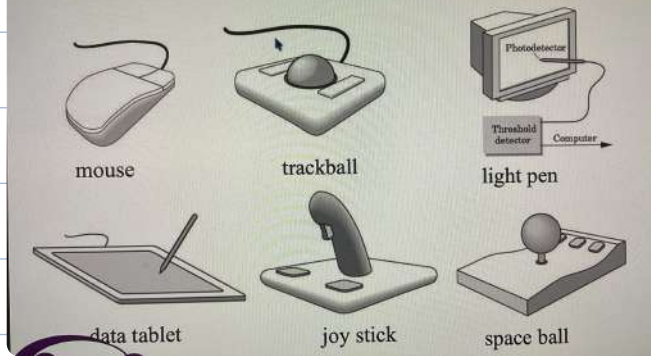
Project Sketchpad

- Ivan Sutherland (MIT 1963) established the basic interactive paradigm that characterizes interactive computer graphics:
 - User sees an *object* on the display
 - User points to (*picks*) the object with an input device (light pen, mouse, trackball)
 - Object changes (moves, rotates, morphs)
 - Repeat

Graphical Input

- Devices can be described either by
 - Physical properties
 - Mouse
 - Keyboard
 - Trackball
 - Logical Properties
 - What is returned to program via API
 - A position
 - An object identifier
- Modes
 - How and when input is obtained
 - Request or event

Physical Devices



Incremental (Relative) Devices

- Devices such as the data tablet return a position directly to the operating system
- Devices such as the mouse, trackball, and joy stick return incremental inputs (or velocities) to the operating system
 - Must integrate these inputs to obtain an absolute position
 - Rotation of cylinders in mouse
 - Roll of trackball
 - Difficult to obtain absolute position
 - Can get variable sensitivity

Logical Devices

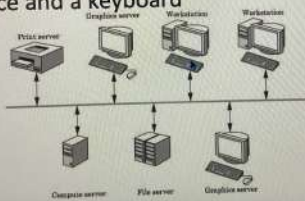
- Consider the C and C++ code
 - C++: `cin >> x;`
 - C: `scanf ("%d", &x);`
- What is the input device?
 - Can't tell from the code
 - Could be keyboard, file, output from another program
- The code provides *logical input*
 - A number (an `int`) is returned to the program regardless of the physical device

Graphical Logical Devices

- Graphical input is more varied than input to standard programs which is usually numbers, characters, or bits
- Two older APIs (GKS, PHIGS) defined six types of logical input
 - **Locator**: return a position
 - **Pick**: return ID of an object
 - **Keyboard**: return strings of characters
 - **Stroke**: return array of positions
 - **Valuator**: return floating point number
 - **Choice**: return one of n items

X Window Input

- The X Window System introduced a client-server model for a network of workstations
 - **Client**: OpenGL program
 - **Graphics Server**: bitmap display with a pointing device and a keyboard

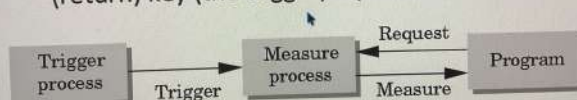


Input Modes

- Input devices contain a *trigger* which can be used to send a signal to the operating system
 - Button on mouse
 - Pressing or releasing a key
- When triggered, input devices return information (their *measure*) to the system
 - Mouse returns position information
 - Keyboard returns ASCII code

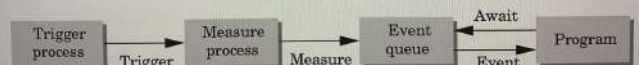
Request Mode

- Input provided to program only when user triggers the device
- Typical of keyboard input
 - Can erase (backspace), edit, correct until enter (return) key (the trigger) is pressed



Event Mode

- Most systems have more than one input device, each of which can be triggered at an arbitrary time by a user
- Each trigger generates an *event* whose measure is put in an *event queue* which can be examined by the user program



Event Types

- Window: resize, expose, iconify
- Mouse: click one or more buttons
- Motion: move mouse
- Keyboard: press or release a key
- Idle: no event
 - Define what should be done if no other event is in queue

CALLBACKS:

- programming interface for event-driven input
 - define a callback function for each type of event the graphics system recognizes
 - this user supplied function is executed when the event occurs
- eg:

`glutMouseFunc(my mouse)`

- GLUT recognizes a subset of events recognized by any particular window system (Windows, X, Macintosh)

(1) `glutDisplayFunc`

(2) `glutMouseFunc`

(3) `glutReshapeFunc`

(4) `glutKeyboardFunc`

(5) `glutIdleFunc`

(6) `glutMotionFunc`

(7) `glutPassiveMotionFunc`

⇒ GLUT Event Loop:

→ last line in main.c must be `glutMainLoop();`



puts the program in an ∞ event loop

→ in each pass through the event loop, GLUT

- looks at the events in queue
- for each event in the queue, GLUT executes the appropriate call back func if defined
- if no callback is defined for event, it is ignored.

The display callback

- The display callback is executed whenever GLUT determines that the window should be refreshed, for example
 - When the window is first opened
 - When the window is reshaped
 - When a window is exposed
 - When the user program decides it wants to change the display
- In **main.c**
 - **glutDisplayFunc(mydisplay)** identifies the function to be executed
 - Every GLUT program must have a display callback

Posting redisplay

- Many events may invoke the display callback function
 - Can lead to multiple executions of the display callback on a single pass through the event loop

- We can avoid this problem by instead using

`glutPostRedisplay()` ;

which sets a flag.

- GLUT checks to see if the flag is set at the end of the event loop
- If set then the display callback function is executed

The mouse callback

`glutMouseFunc(mymouse)`

`void mymouse(GLint button, GLint state, GLint x, GLint y)`

- Returns
 - which button (`GLUT_LEFT_BUTTON`, `GLUT_MIDDLE_BUTTON`, `GLUT_RIGHT_BUTTON`) caused event
 - state of that button (`GLUT_UP`, `GLUT_DOWN`)
 - Position in window

Positioning

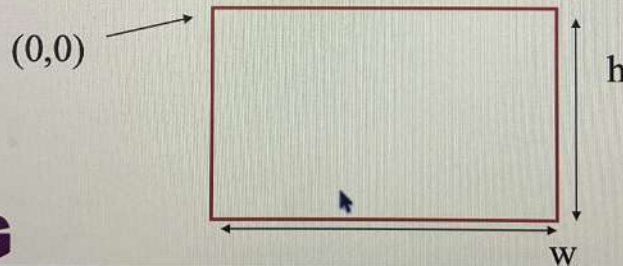
The position in the screen window is usually measured in pixels with the origin at the top-left corner

Consequence of refresh done from top to bottom

OpenGL uses a world coordinate system with origin at the bottom left

Must invert y coordinate returned by callback by height of window

$$y = h - y;$$



130

Terminating a program

- We can use the simple mouse callback

```
void mouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_RIGHT_BUTTON && state==GLUT_DOWN)
        exit(0);
}
```

Using the mouse position

- In the next example, we draw a small square at the location of the mouse each time the left mouse button is clicked
- This example does not use the display callback but one is required by GLUT; We can use the empty display callback function

```
mydisplay() {}
```

Drawing squares at cursor location

```
void mymouse(int btn, int state, int x, int y)
{
    if(btn==GLUT_RIGHT_BUTTON && state==GLUT_DOWN)
        exit(0);
    if(btn==GLUT_LEFT_BUTTON && state==GLUT_DOWN)
        drawSquare(x, y);
}
void drawSquare(int x, int y)
{
    y=w-y; /* invert y position */
    glColor3ub( (char) rand()%256, (char) rand()%256,
                (char) rand()%256); /* a random color */
    glBegin(GL_POLYGON);
        glVertex2f(x+size, y+size);
        glVertex2f(x-size, y+size);
        glVertex2f(x-size, y-size);
        glVertex2f(x+size, y-size);
    glEnd();
}
```

Using the motion callback

- We can draw squares (or anything else) continuously as long as a mouse button is pressed by using the motion callback
 - `glutMotionFunc(drawSquare)`
- We can draw squares without pressing a button using the passive motion callback
 - `glutPassiveMotionFunc(drawSquare)`

Using the keyboard

```
glutKeyboardFunc (mykey)
```

```
void mykey(unsigned char key,  
            int x, int y)
```

- Returns ASCII code of key pressed and mouse location

```
void mykey()
```

```
{
```

```
    if(key == 'Q' | key == 'q')
```

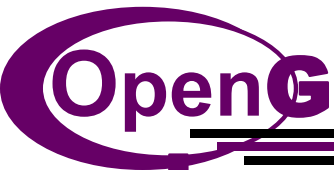
```
        exit(0);
```

Special and Modifier Keys

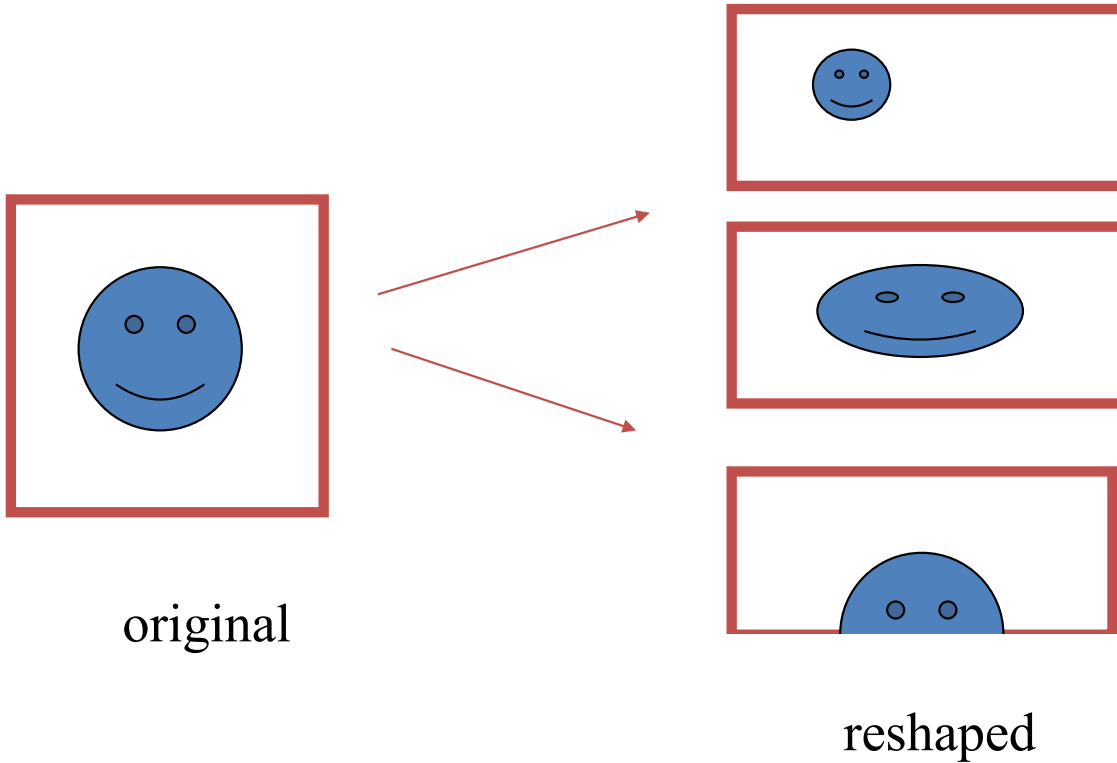
- GLUT defines the special keys in `glut.h`
 - Function key 1: `GLUT_KEY_F1`
 - Up arrow key: `GLUT_KEY_UP`
 - `if (key == 'GLUT_KEY_F1' .....`
- Can also check of one of the modifiers
 - `GLUT_ACTIVE_SHIFT`
 - `GLUT_ACTIVE_CTRL`
 - `GLUT_ACTIVE_ALT`is pressed by
`glutGetModifiers ()`
 - Allows emulation of three-button mouse with one- or two-button mice

Reshaping the window

- We can reshape and resize the OpenGL display window by pulling the corner of the window
- What happens to the display?
 - Must redraw from application
 - Two possibilities
 - Display part of world
 - Display whole world but force to fit in new window
 - Can alter aspect ratio



Reshape possibilities



The Reshape callback

```
glutReshapeFunc(myreshape)
```

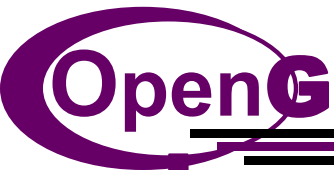
```
void myreshape( int w, int h)
```

- Returns width and height of new window (in pixels)
- A redisplay is posted automatically at end of execution of the callback
- GLUT has a default reshape callback but you probably want to define your own
- The reshape callback is good place to put viewing functions because it is invoked when the window is first opened

Example Reshape

- This reshape preserves shapes by making the viewport and world window have the same aspect ratio

```
void myReshape(int w, int h)
{
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION); /* switch matrix mode */
    glLoadIdentity();
    if (w <= h)
        gluOrtho2D(-2.0, 2.0, -2.0 * (GLfloat) h / (GLfloat) w,
                    2.0 * (GLfloat) h / (GLfloat) w);
    else gluOrtho2D(-2.0 * (GLfloat) w / (GLfloat) h, 2.0 *
                    (GLfloat) w / (GLfloat) h, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW); /* return to modelview mode */
}
```



Picking

- Identify a user-defined object on the display
- In principle, it should be simple because the mouse gives the position and we should be able to determine to which object(s) a position corresponds
- Practical difficulties
 - Pipeline architecture is feed forward, hard to go from screen back to world
 - Complicated by screen being 2D, world is 3D
 - How close do we have to come to object to say we selected it?

Three Approaches

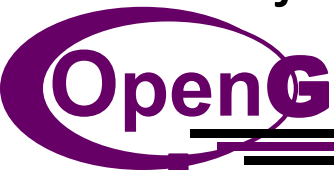
- Hit list
 - Most general approach but most difficult to implement
- Use back or some other buffer to store object ids as the objects are rendered
- Rectangular maps
 - Easy to implement for many applications

Rendering Modes

- OpenGL can render in one of three modes selected by `glRenderMode(mode)`
 - `GL_RENDER`: normal rendering to the frame buffer (default)
 - `GL_FEEDBACK`: provides list of primitives rendered but no output to the frame buffer
 - `GL_SELECTION`: Each primitive in the view volume generates a *hit record* that is placed in a *name stack* which can be examined later

Selection Mode Functions

- `glSelectBuffer()`: specifies name buffer
 - `glInitNames()`: initializes name buffer
 - `glPushName(id)`: push id on name buffer
 - `glPopName()`: pop top of name buffer
 - `glLoadName(id)`: replace top name on buffer
-
- id is set by application program to identify objects



Display Lists

- Conceptually similar to a graphics file
 - Must define (name, create)
 - Add contents
 - Close
- In client-server environment, display list is placed on server
 - Can be redisplayed without sending primitives over network each time

Display List Functions

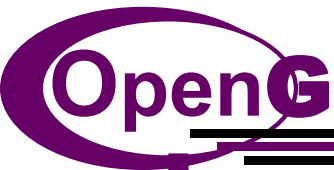
- Creating a display list

```
GLuint id;
```

```
void init()  
{  
    id = glGenLists( 1 );  
    glNewList( id, GL_COMPILE );  
    /* other OpenGL routines */  
    glEndList();  
}
```

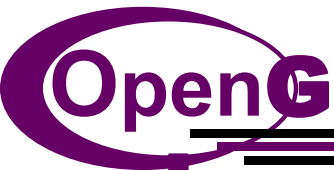
- Call a created list

```
void display()  
{  
    glCallList( id );  
}
```



Display Lists and State

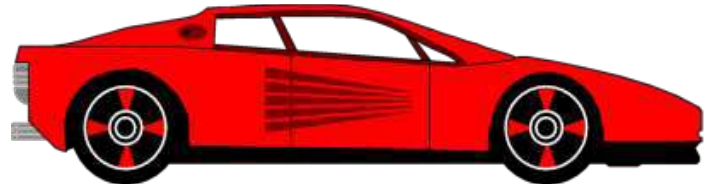
- Most OpenGL functions can be put in display lists
- State changes made inside a display list persist after the display list is executed
- Can avoid unexpected results by using `glPushAttrib` and `glPushMatrix` upon entering a display list and `glPopAttrib` and `glPopMatrix` before exiting



Hierarchy and Display Lists

- Consider model of a car
 - Create display list for chassis
 - Create display list for wheel

```
glNewList( CAR, GL_COMPILE );  
    glCallList( CHASSIS );  
    glTranslatef( ... );  
    glCallList( WHEEL );  
    glTranslatef( ... );  
    glCallList( WHEEL );  
    ...  
glEndList();
```



Animating a Display

- When we redraw the display through the display callback, we usually start by clearing the window
 - `glClear()`

then draw the altered display

- Problem: the drawing of information in the frame buffer is decoupled from the display of its contents
 - Graphics systems use dual ported memory
- Hence we can see partially drawn display
 - See the program `single_double.c` for an example with a rotating cube

Double Buffering

- Instead of one color buffer, we use two
 - **Front Buffer**: one that is displayed but not written to
 - **Back Buffer**: one that is written to but not displayed
- Program then requests a double buffer in main.c
 - `glutInitDisplayMode(GL_RGB | GL_DOUBLE)`
 - At the end of the display callback buffers are swapped

```
void mydisplay()  
{  
    glClear(GL_COLOR_BUFFER_BIT|...)  
    .  
    /* draw graphics here */  
    .  
    glutSwapBuffers()
```

Using the idle callback

- The idle callback is executed whenever there are no events in the event queue

- `glutIdleFunc(myidle)`

- Useful for animations

```
void myidle() {  
    /* change something */  
    t += dt  
    glutPostRedisplay();  
}
```

```
Void mydisplay() {  
    glClear();  
    /* draw something that depends on t */  
    glutSwapBuffers();  
}
```

Using globals

- The form of all GLUT callbacks is fixed
 - `void mydisplay()`
 - `void mymouse(GLint button, GLint state, GLint x, GLint y)`
- Must use globals to pass information to callbacks

```
float t; /*global */
```

```
void mydisplay()
```

```
{
```

```
/* draw something that depends on t
```

```
}
```

Toolkits and Widgets

- Most window systems provide a toolkit or library of functions for building user interfaces that use special types of windows called *widgets*
- Widget sets include tools such as
 - Menus
 - Slidebars
 - Dials
 - Input boxes
- But toolkits tend to be platform dependent
- GLUT provides a few widgets including menus

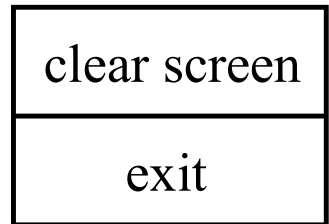
Menus

- GLUT supports pop-up menus
 - A menu can have submenus
- Three steps
 - Define entries for the menu
 - Define action for each menu item
 - Action carried out if entry selected
 - Attach menu to a mouse button

Defining a simple menu

- In `main.c`

```
menu_id = glutCreateMenu(mymenu);  
glutAddmenuEntry("clear Screen", 1);  
  
gluAddMenuEntry("exit", 2);  
  
glutAttachMenu(GLUT_RIGHT_BUTTON);
```



entries that appear when
right button pressed

identifiers

Menu actions

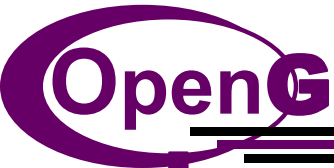
- Menu callback

```
void mymenu(int id)
{
    if(id == 1) glClear();
    if(id == 2) exit(0);
}
```

- Note each menu has an id that is returned when it is created
- Add submenus by

```
glutAddSubMenu(char *submenu_name, submenu id)
```

entry in parent menu



Other functions in GLUT

- Dynamic Windows
 - Create and destroy during execution
- Subwindows
- Multiple Windows
- Changing callbacks during execution
- Timers
- Portable fonts
 - `glutBitmapCharacter`
 - `glutStrokeCharacter`

1.1 THE THREE I's OF VIRTUAL REALITY

It is clear from the foregoing description that virtual reality is both interactive and immersive. These features are the two I's that most people are familiar with. There is, however, a third feature of virtual reality that fewer people are aware of. Virtual reality is not just a medium or a high-end user interface, it also has applications that involve solutions to real problems in engineering, medicine, the military, etc. These applications are designed by virtual reality developers. The extent to which an application is able to solve a particular problem, that is, the extent to which a simulation performs well, depends therefore very much on the human *imagination*, the third "I" of VR. Virtual reality is therefore an integrated trio of immersion–interaction–imagination, as shown in Figure 1.2. The imagination part of VR refers also to the mind's capacity to perceive nonexistent things. The triangle in Figure 1.2, for example, is easily "seen" by the reader, yet it only exists in his or her imagination.

1.2 A SHORT HISTORY OF EARLY VIRTUAL REALITY

Virtual reality is not a new invention, but dates back more than 40 years. In 1962, U.S. Patent #3,050,870 was issued to Morton Heilig for his invention entitled *Sensorama Simulator*, which was the first virtual reality video arcade. As shown in Figure 1.3, this early virtual reality workstation had three-dimensional (3D) video feedback (obtained with a pair of side-by-side 35-mm cameras), motion, color, stereo sound, aromas, wind effects (using small fans placed near the user's head), and a seat that vibrated. It was thus possible to simulate a motorcycle ride through New York, where the "rider" sensed the wind and felt the pot-holes of the road as the seat vibrated. The rider could even smell food when passing by a store.

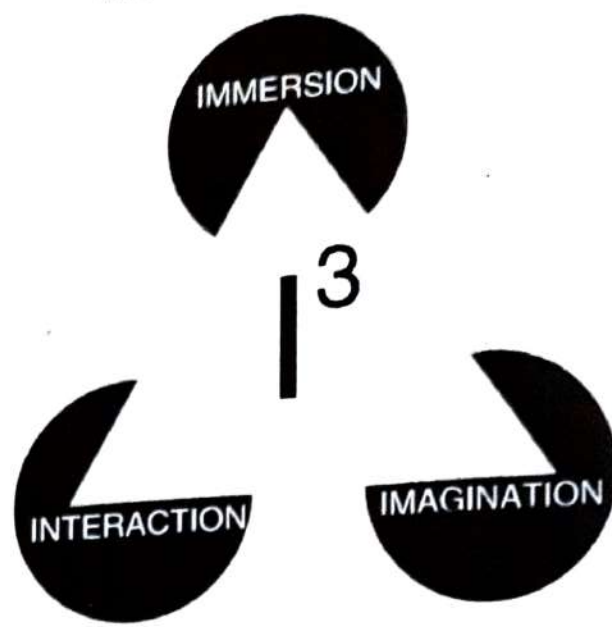


Fig. 1.2 The three I's of virtual reality, immersion–interaction–imagination. Adapted from Burdea [1993]. Reprinted by permission.

Heilig also realized the possibilities of head-mounted television. He designed a simulation mask that included 3D slides with wide peripheral effects, focusing controls and optics, stereophonic sound, and the capability to include smell. A depiction from U.S. Patent #2,955,156 issued to him on October 4, 1960, is shown in Figure 1.4. Heilig, a cinematographer by profession, was well ahead of his time. Like Jules Verne, he imagined a new machine that would replace the classical cinematographic experience of today. He was also like Thomas Edison, an inventor who not only dreamed ideas, but also transformed them into real machines. At the time of Heilig's inventions, nobody realized the revolutionary technological progress they represented.

Heilig's initial work on head-mounted displays (HMD) was continued by Ivan Sutherland. In 1966 Sutherland used two cathode ray tubes (CRTs) mounted along the user's ears. Today's high-end HMDs use miniature CRTs mounted in the same configuration. Since the tubes available in 1966 were much heavier than those in use today, Sutherland had to rely on a mechanical arm to support the weight of the display. This mechanical arm had potentiometers that measured the user's view direction. Most of today's HMDs use noncontact position tracking (magnetic or ultrasound), but this technology was not available in the 1960s.

While working on his head-mounted display, Sutherland realized that he could use computer-generated scenes instead of analog images taken by cameras, and began to design such a scene generator. This was the precursor of the modern graphics accelerator, a key part of VR hardware. The computer generates a sequence of scenes each a bit different and each displayed in a fraction of a second. The overall effect is that of animations such as flyby simulations used to train pilots. Early graphics scene generators produced by Evans and Sutherland around 1973 could display simple scenes of only 200–400 polygons. Each scene took about 1/20 sec to compute and display, so that about 20 scenes (or frames) were displayed every second. More complex scenes were made of more polygons, took a longer time to display, and therefore consisted of fewer frames per second. As a consequence, animation (which requires more than 16 frames/sec to be smooth) suffered.

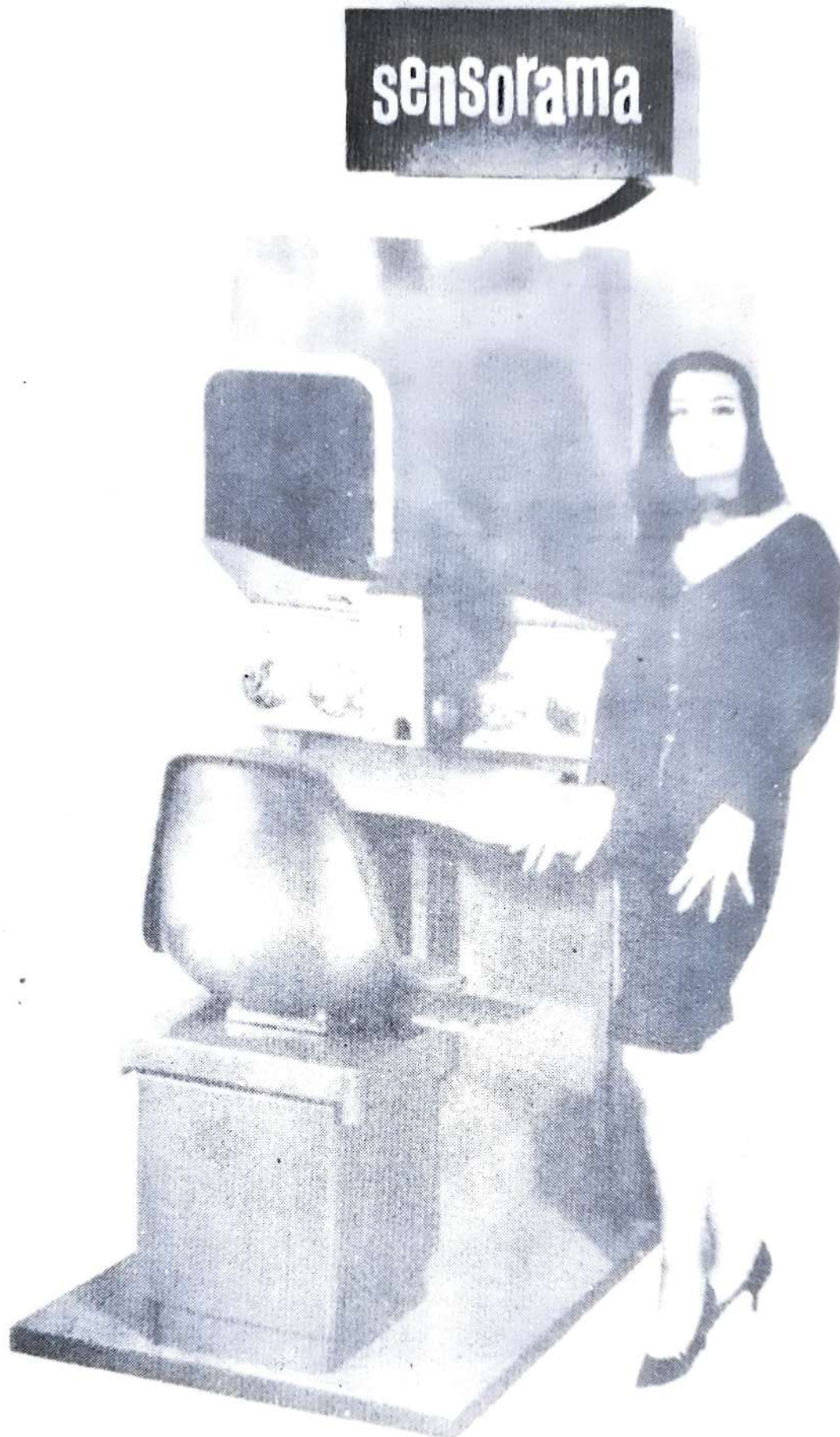
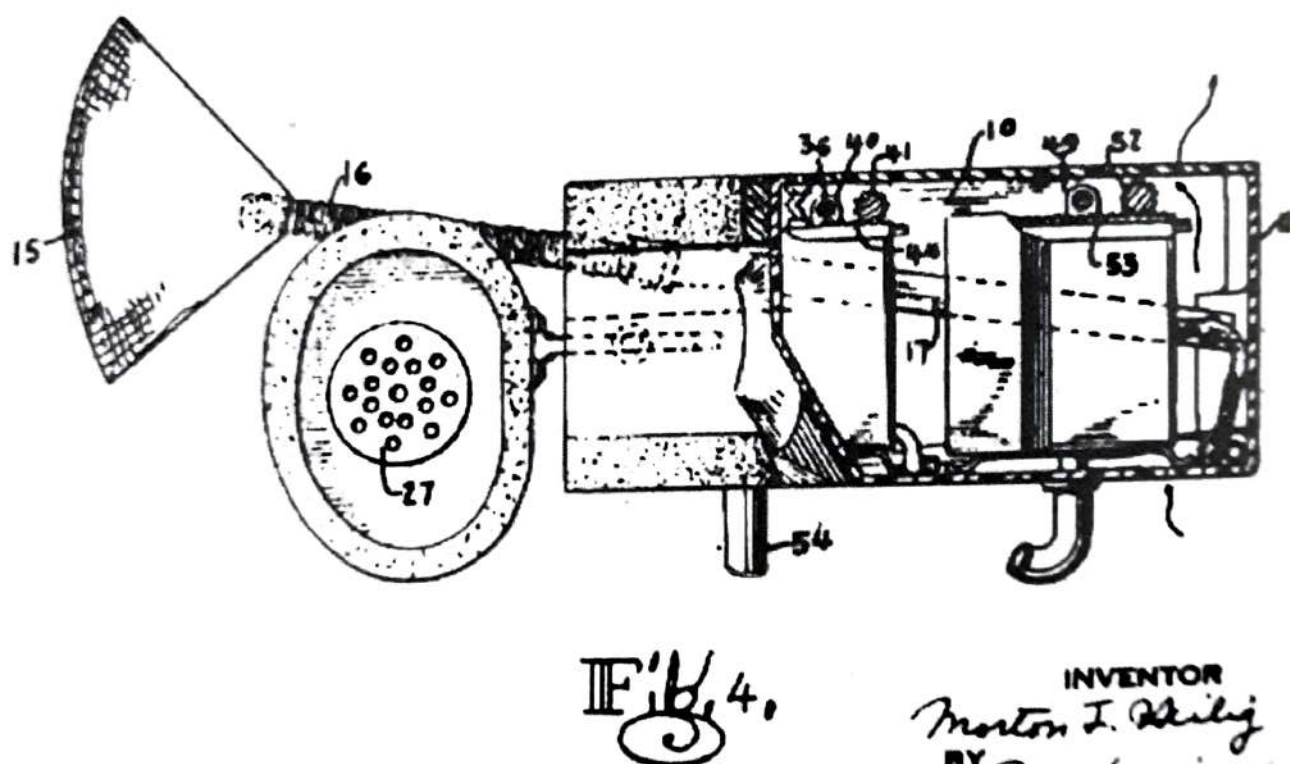
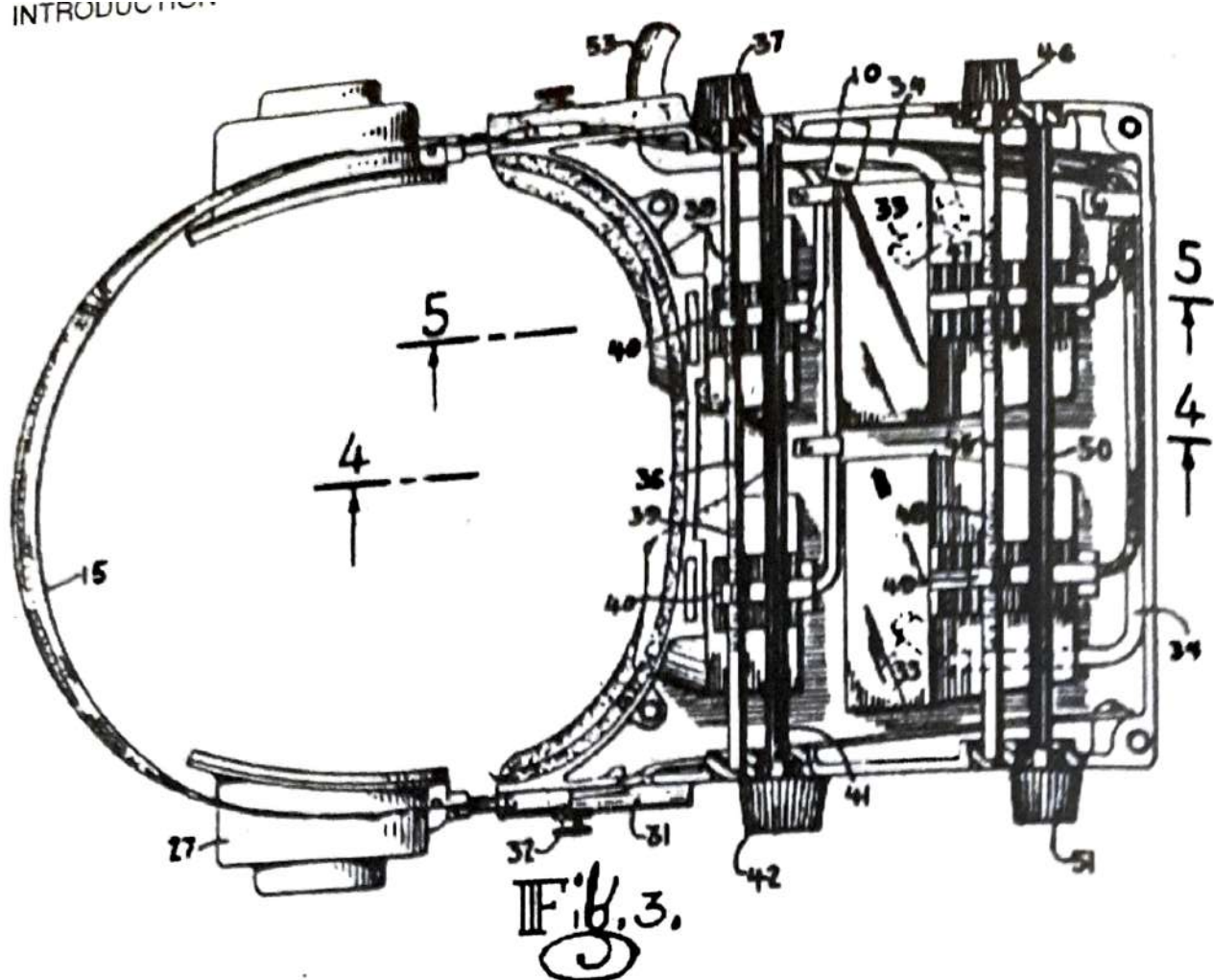


Fig. 1.3 The Sensorama Simulator prototype. Courtesy of M. Heilig.

Sutherland's vision of an "ultimate display" to the virtual world was not limited to graphics. In 1965 he predicted that the sense of touch (or haptics) would be added in order to allow users to feel the virtual objects they saw [Sutherland, 1965]. This idea was made reality by Frederick Brooks, Jr., and his colleagues at the University of North Carolina at Chapel Hill. By 1971 these scientists demonstrated the ability to



INVENTOR
Morton J. Heilig
BY *Brooklyn*
ATTORNEYS

Fig. 1.4 Heilig's early head-mounted display patent. From Heilig [1960]. Reprinted by permission.

simulate two-dimensional continuous force fields associated with molecular docking forces [Batter and Brooks, 1971]. Later they simulated three-dimensional collision forces using a surplus robotic arm normally used in nuclear material handling. Most of today's haptic technology is based on miniature robotic arms.

The military was very eager to test the new digital simulators, since they hoped to replace very expensive analog ones. Flight simulators were hardware designed for a particular airplane model. When that airplane became obsolete, so did its simulator, and this was a constant drain of funds. If the simulation could be done in software

on a general-purpose platform, then a change in airplane models would only require software upgrades. The advantage seems obvious. Much research went on in the 1970s and early 1980s on flight helmets and modern simulators for the military, but much of this work was classified and was not published. This changed when funds for defense were cut and some researchers migrated to the civilian sector.

The National Aeronautics and Space Agency (NASA) was another agency of the American government interested in modern simulators. It needed simulations for astronaut training, as it was difficult or impossible to otherwise recreate conditions existing in outer space or on distant planets. In 1981, on a very small budget, NASA created the prototype of a liquid crystal display (LCD)-based HMD, which they named the Virtual Visual Environment Display (VIVED). NASA scientists simply disassembled commercially available Sony Watchman TVs and put the LCDs on special optics. These optics were needed to focus the image close to the eyes without effort. The majority of today's virtual reality head-mounted displays still use the same principle. NASA scientists then proceeded to create the first virtual reality system by incorporating a DEC PDP 11-40 host computer, a Picture System 2 graphics computer (from Evans and Sutherland), and a Polhemus noncontact tracker. The tracker was used to measure the user's head motion and transmit it to the PDP 11-40. The host computer then relayed these data to the graphics computer, which calculated new images displayed in stereo on the VIVED.

In 1985 the project was joined by Scott Fisher, who integrated a new kind of sensing glove into the simulation. The glove was developed earlier by Thomas Zimmerman and Jaron Lanier as a virtual programming interface for nonprogrammers. A photo of Fisher experimenting with the VIVED system is shown in Figure 1.5. By 1988 Fisher and Elizabeth Wenzel created the first hardware capable of manipulating up



Fig. 1.5 NASA VIVED prototype. Courtesy of Telepresence Research Inc.

to four 3D virtual sound sources. These are sounds that remain localized in space even when the user turns his or her head. This represented a very powerful addition to the simulation. The original VIVED project became VIEW (for Virtual Interface Environment Workstation) and the original software was ported to a newer Hewlett-Packard 9000, which had sufficient graphics performance to replace the wireframe rendering used in VIVED with more realistic flat-shaded surfaces.

With all the aforementioned technological developments, scientific exchange of information among the small group of specialists of the time followed. France was one of the first countries to organize a major international conference on the subject, held in Montpellier in March 1992. The name of this conference was Interfaces for Real and Virtual Worlds, and it drew hundreds of papers and many vendors. Later the same year the United States organized the first conference on Medicine Meets Virtual Reality. In San Diego, about 180 medical practitioners met with 60 scientists and engineers to discuss the great potential of virtual reality as a tool for medicine. In September 1993 the world's largest professional society, the Institute of Electrical and Electronics Engineers (IEEE), organized its first VR conference in Seattle. Virtual reality had become part of the mainstream scientific and engineering community.

1.3 EARLY COMMERCIAL VR TECHNOLOGY

The first company to sell VR products was VPL Inc., headed by Jaron Lanier. Until its demise in 1992 this company produced the first sensing glove, called the DataGlove (Figure 1.6a) [VPL, 1987]. The standard interfaces of the time (and still today) were the keyboard and the mouse. Compared to these, the VPL DataGlove represented a quantum improvement in the natural way one could interact with computers. Its fiber-optic sensors allowed computers to measure finger and thumb bending, and thus interaction was possible through gestures. Its drawbacks were high price (thousands of dollars), lack of tactile feedback, and difficulty in accommodating different hand sizes.

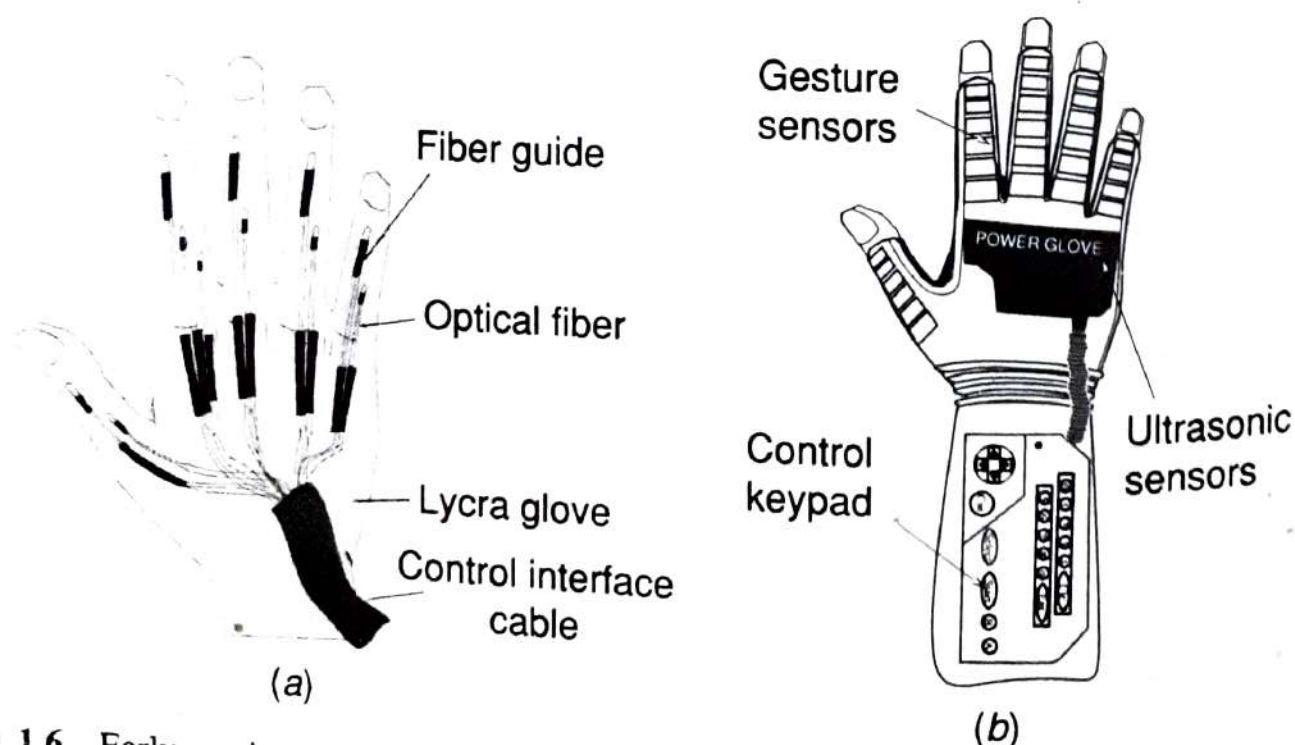


Fig. 1.6 Early sensing glove technology: (a) the VPL DataGlove; (b) the PowerGlove. From Burdea [1993]. Reprinted by permission.

Shortly after the appearance of the VPL DataGlove, the game company Nintendo introduced the much cheaper PowerGlove, shown in Figure 1.6*b* [Burdea, 1993]. It used ultrasonic sensors to measure wrist position relative to the PC screen and conductive ink flex sensors to measure finger bending. In 1989 almost one million such new game consoles were sold in a consumer frenzy that was later repeated with the introduction of Sony Play Station. The downfall of the PowerGlove was lack of sufficient games that used it, such that by 1993 its production had stopped.

The first commercial head-mounted displays, called EyePhones, were introduced by VPL in the late 1980s. These HMDs used LCD displays to produce a stereo image, but at extremely low resolution (360×240 pixels), such that virtual scenes appeared blurred. Other drawbacks were high price (\$11,000 each) and large weight (2.4 kg).

Researchers now had an initial set of specialized hardware with which to start developing applications. However, they first had to solve various integration issues as well as develop most of the required software from scratch. The idea of a turnkey VR system originated with VPL as well. Its RB2 Model 2 offered a rack assembly housing the EyePhone HMD interface, the VPL DataGlove Model 2 electronic unit, a spatial tracking unit for the HMD, a design and control workstation, as well as connections to an SGI 4D/310 VGX graphics renderer and to an optional 3D sound system.

The next step in integration was to shrink each of these components and put them on a board in a single desk-side cabinet. In early 1991 a company in the United Kingdom, Division Ltd., introduced the first integrated commercial VR workstation. It was called Vision and was followed by the more powerful Provision 100 [Grimsdale, 1992], which is illustrated in Figure 1.7. The Provision 100 parallel architecture had



Fig. 1.7 The Provision 100 VR workstation. Courtesy of Division Ltd.

multiple processors, stereo display on an HMD, 3D sound, hand tracking, and gesture recognition. The architecture also had an input/output (I/O) card and was scalable, allowing additional I/O processors to be added. Its i860 dedicated geometry processor with a custom polygon accelerator provided 35,000 Gouraud-shaded and Z-buffered polygons per second. This was a clear improvement over the speed of the HP 9000 computer used in NASA's VIEW system, but it came at a high price (\$70,000).

Although turnkey hardware systems appeared in early 1990s, the VR software development and debugging time continued to be a problem. In 1992, the small U.S. company Sense8 Co. developed the first version of its WorldToolKit (WTK) [Sense8 Co., 1992], a library of C functions written specifically for VR applications. With this toolkit developing VR software became more like a science, and debugging time was reduced significantly.

Another popular toolkit of the 1990s was the Virtual Reality Toolkit (VRT3), developed in the United Kingdom by Dimension International (the company later became Superscape PLC) [Dimension International, 1993]. Unlike WTK, VRT3 was designed to run on multiple computing platforms without the need for (then) pricey graphics accelerators. Also, unlike WTK, which is text-based, VRT3 used graphical programming through menus and icons. This made programming easier to learn, but less rich in functionality, owing to the limited menu of available functions and the limited number of supported interfaces.

1.5 THE FIVE CLASSIC COMPONENTS OF A VR SYSTEM

The discussion in this book will focus on the five classic components of a VR system, as depicted in Figure 1.11 [Burdea and Coiffet, 1993]. Chapters 2 and 3 concentrate

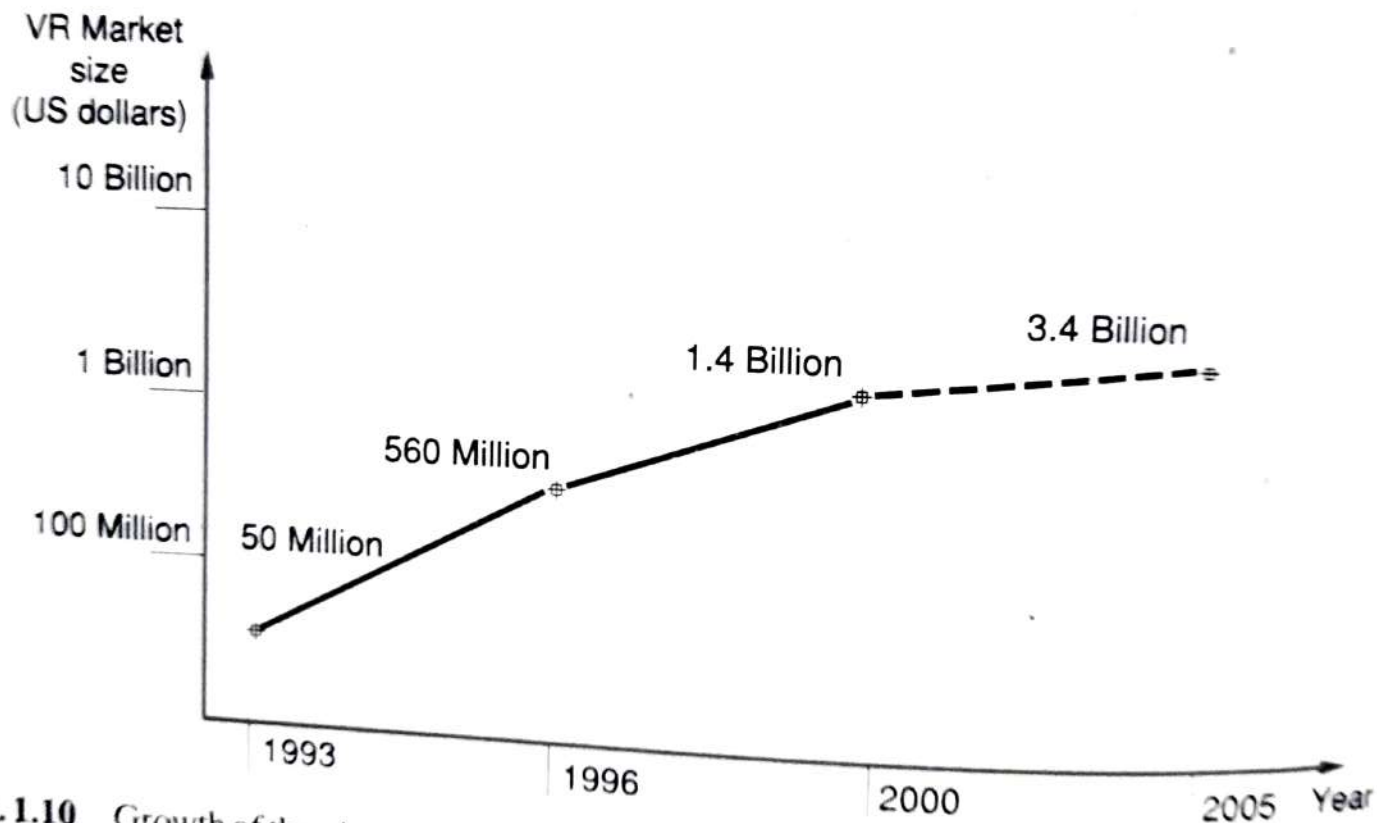


Fig. 1.10 Growth of the virtual reality industry since 1993. Based on Donovan [1993], Delaney [1997], and Machover [2000].

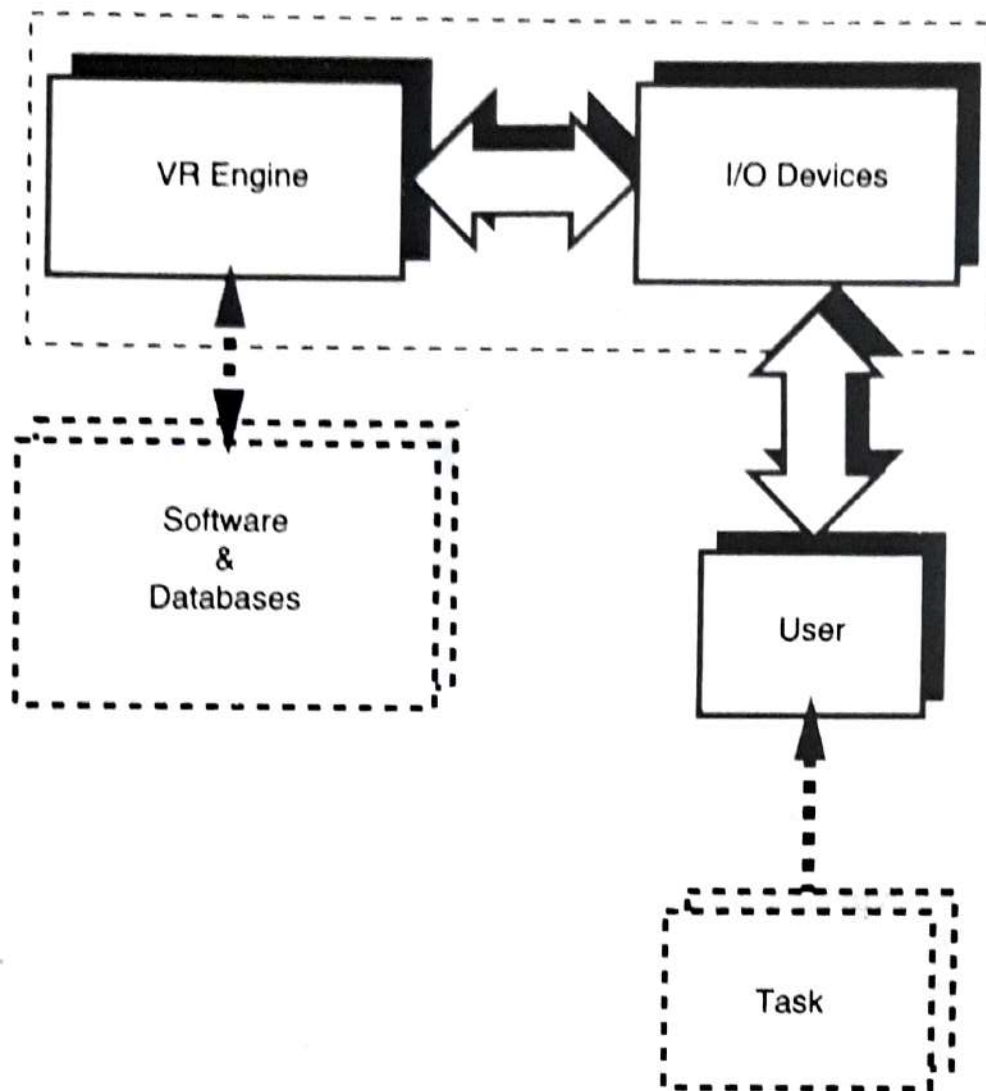


Fig. 1.11 The five classic components of a VR system. From Burdea and Coiffet [1993]. © Editions Hermes. Reprinted by permission.

on the very important I/O devices used either for user input (such as trackers, gloves, or mice) or output (such as HMDs, large-volume displays, force feedback robotic arms, etc.). The special-purpose computer architecture designed to match the high I/O and computation demands of real-time VR simulations is presented in Chapter 4. Chapter 5 deals with software for virtual object modeling, involving its geometry, texture, intelligent behavior, and physical modeling of hardness, inertia, surface plasticity, etc. Chapter 6 reviews a number of powerful programming packages, such as WorldToolKit and Java 3D, designed to help the virtual reality application developer. In Chapter 7 we analyze several human factors issues affecting simulation efficiency as well as user comfort and safety. Chapter 8 discusses traditional VR applications. These integrate all the aspects discussed in earlier chapters and make VR a tool for solving various practical problems in medical care, education, arts, entertainment, and the military. A look at emerging VR applications in manufacturing, robotics, and information visualization makes up the subject of Chapter 9, which concludes the book.

1.6 REVIEW QUESTIONS

1. What is virtual reality?
2. How does virtual reality differ from augmented reality and telepresence?

3. What are commonalities and differences between virtual reality and 3D computer graphics?
4. What was Heilig's role in the development of VR?
5. What was NASA's role in early VR developments and why was it interested?
6. What were the first commercial VR products?
7. What happened with the VR industry in the 1990s?
8. What are the five classic components of a VR system?